



Impact of optimization scope on solution quality and stability in dynamic flexible job shop rescheduling

Ruirui Sun^{a,b}, Guanghe Cheng^{a,b,*}, Qingyan Ding^{a,b}, Xiaojie Zhao^c

^a Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center(National Supercomputing Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), 19 Keyuan Road, Jinan, 250014, China

^b Shandong Provincial Key Laboratory of Industrial Network and Information System Security, Shandong Fundamental Research Center for Computer Science, 19 Keyuan Road, Jinan, 250014, China

^c Qingdao Economic and Technological Development Zone Haier Water Heater Co., 1 Haier Road, Qingdao, 266100, China

ARTICLE INFO

Keywords:

Dynamic flexible job shop scheduling
Rescheduling
Optimization scope
Machine breakdown
Scheduling trade-offs

ABSTRACT

In dynamic manufacturing environments, disruptions impose a critical trade-off between locally repairing the schedule to preserve stability and globally re-optimizing it to recover efficiency. This study addresses this problem in Dynamic Flexible Job Shop Scheduling (DFJSP) by analyzing the impact of the *Optimization Scope*—the subset of operations selected for rescheduling. We introduce a formal framework defining four hierarchical optimization scopes, ranging from minimal repair to global re-optimization. Experimental results identify a “Greedy Trap” in myopic strategies (Ω_G), where localized optimization triggers cascading delays and schedule instability, evidenced by a high Relative Standard Deviation of RPD (12.38%). To address this, we propose the Event-driven Rescheduling with Elite Evolution (ERRE) strategy, which operates on the Affected Scope (Ω_A). An ablation study confirms that the performance gains stem specifically from this scope definition mechanism rather than the underlying algorithm. Contrary to the assumption that global search yields superior quality, results show that under strict real-time constraints, ERRE achieves the lowest Mean-RPD (0.76%) across all scenarios. It significantly outperforms the global rescheduling baseline (4.15%), which is limited by search convergence within short decision windows. By effectively balancing solution quality with computational efficiency, ERRE provides a robust solution suitable for integration into Manufacturing Execution Systems (MES).

1. Introduction

The Flexible Job Shop Scheduling Problem (FJSP) occupies a central role in the production management of modern discrete manufacturing systems. By extending the classical Job Shop Problem (JSP) to allow operations to be processed on a set of alternative machines, FJSP introduces a critical layer of flexibility. This characteristic, originally formulated by Brandimarte (1993) and recently surveyed by Dauzère-Pérès et al. (2024), enhances system agility and robustness, catering specifically to high-mix, low-volume production demands. In advanced manufacturing environments, the scheduling problem is further complicated by the integration of realistic constraints, such as sequence-dependent setup times and position-dependent effects (Angel-Bello et al., 2020; Gupta & Jain, 2022). While these factors improve model fidelity, they significantly expand the decision space and create a tight coupling between machine assignment and operation sequencing. Consequently,

FJSP represents a canonical NP-hard combinatorial optimization problem, necessitating highly efficient algorithmic solutions as discussed in the systematic review by Jiang et al. (2023).

Although numerous algorithms have been proposed for static FJSP, the utility of offline-generated schedules is often constrained by the stochastic nature of the shop floor. In practice, static schedules are short-lived due to unpredictable events such as machine breakdowns, urgent order arrivals, and material delays (Ouelhadj & Petrovic, 2009; Priore et al., 2014). These disruptions can rapidly invalidate pre-computed plans or degrade their optimality. Consequently, the operational challenge shifts from static optimization to dynamic rescheduling: the task of generating a new schedule within a constrained time frame that balances performance metrics against system stability. This constitutes a critical real-time decision-making function in modern Manufacturing Execution Systems (MES) (Vieira et al., 2003).

* Corresponding author at: Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center(National Supercomputing Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), 19 Keyuan Road, Jinan, 250014, China.

E-mail addresses: b1043124018@stu.qilu.edu.cn (R. Sun), chenggh@qilu.edu.cn (G. Cheng), dingqy@sdas.org (Q. Ding), zhaoxiaojie@haier.com (X. Zhao).

<https://doi.org/10.1016/j.cie.2026.111943>

Received 21 October 2025; Received in revised form 21 February 2026; Accepted 25 February 2026

Available online 26 February 2026

0360-8352/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

To address dynamic scheduling, existing literature has predominantly focused on the algorithmic methodology—i.e., “how to reschedule”. This has led to the development of sophisticated solvers, ranging from metaheuristics like particle swarm optimization for simultaneous assignment and sequencing (Zarrouk et al., 2019), to strategies explicitly handling setup constraints to maintain stability (Angel-Bello et al., 2021). Recently, Artificial Intelligence (AI) techniques, particularly Deep Reinforcement Learning (DRL), have gained prominence for learning dynamic scheduling policies. Approaches vary from using Double DQN with attention mechanisms for machine breakdowns (Wu et al., 2025), to Proximal Policy Optimization (PPO) for digital twin control (Serrano-Ruiz et al., 2024) or direct job selection (Zhao et al., 2024). Hybrid approaches combining DRL with metaheuristics have also been explored to enhance search efficiency, such as adaptive parameter tuning (Ma et al., 2025) or operator selection in memetic algorithms (Zhang et al., 2024). Furthermore, the integration of Digital Twin (DT) technologies with DRL provides high-fidelity environments for adaptive scheduling (Chang et al., 2023; Fang et al., 2023). Despite these algorithmic advancements, a fundamental strategic question remains under-explored: the determination of the optimal rescheduling magnitude. Specifically, when a disruption occurs, identifying the precise portion of the future plan to re-optimize—defined here as the *optimization scope*—is a critical decision variable that governs the trade-off between computational complexity, solution quality, and stability.

Schedule stability, characterized by the deviation between the new and original schedules, is a pivotal performance dimension. Excessive modifications can induce shop-floor nervousness, resulting in operator confusion, increased setup costs, and operational risks (Leus & Herroelen, 2007; Zhang & Wong, 2017). The selection of the Optimization Scope directly governs this stability. Minimal-scope strategies, such as Right-Shift Rescheduling (RSR), merely postpone affected operations (Herrmann, 2006; Vieira et al., 2003). While this approach maximizes sequence stability, it forfeits opportunities for quality improvement. Conversely, global rescheduling re-optimizes all unfinished operations, theoretically targeting a global optimum. However, given strictly bounded decision windows, global solvers often fail to converge, incurring high computational costs and causing complete disruption of the planned machine assignments (sequence instability). Between these extremes lies a spectrum of intermediate strategies that remains largely uninvestigated. Most studies adopt a fixed Optimization Scope, limiting adaptability. Thus, this paper addresses the following research question: How can the Optimization Scope be systematically modeled and quantified as a control variable to trade off solution quality, computational efficiency, and schedule stability?

This study adopts a predictive-reactive scheduling framework, where the system responds to disruptions upon detection (Vieira et al., 2003). This paradigm contrasts with proactive robust scheduling (Pleier & Schilp, 2024) or end-to-end graph-based learning approaches that internalize dynamics (Liu et al., 2024, 2025). The reactive nature of this framework renders the real-time adjustment of the Optimization Scope a decisive factor, making the proposed methodology particularly relevant for systems requiring agile responses to environmental dynamics.

To address these challenges, this paper presents the following contributions:

1. **A formal computational framework for the Optimization Scope in dynamic rescheduling.** We propose the first systematic framework to model the Optimization Scope, decomposing this traditionally ambiguous choice into four distinct levels: Zero Scope (Ω_0), Greedy Scope (Ω_G), Affected Scope (Ω_A), and Future Scope (Ω_F). This formalization transforms the scope into a quantifiable variable, providing a theoretical foundation for the comparative analysis of rescheduling strategies.

2. **Identification and characterization of the “Greedy Trap”.** Through extensive experiments, we identify the “Greedy Trap”, a failure mode where myopic rescheduling heuristics focused on short-term repairs paradoxically degrade overall system performance. This finding establishes a design principle for automated scheduling agents, highlighting the necessity of coordinated optimization over purely local fixes.
3. **Development of a robust rescheduling strategy: ERRE (Event-driven Rescheduling with Elite Evolution).** Leveraging the concept of Affected Scope, we propose the ERRE strategy. Experimental results demonstrate that ERRE effectively balances the conflicting objectives of solution quality, efficiency, and operational stability. Compared to global rescheduling, ERRE significantly reduces computational overhead while maintaining high-quality solutions, offering a deployable solution for real-world MES integration.

2. Related work

This section reviews the literature on Dynamic Flexible Job Shop Scheduling (DFJSP). We position our study within the predictive-reactive scheduling paradigm and classify existing rescheduling strategies based on their Optimization Scope. This classification facilitates the identification of critical research gaps, motivating the contributions of this paper.

2.1. Predictive-reactive paradigm

Dynamic scheduling addresses the inherent uncertainties in manufacturing systems. As defined by Ouelhadj and Petrovic (2009), it involves the continuous allocation of resources and sequencing of tasks in response to evolving information. Dynamic events—such as machine breakdowns, urgent order arrivals, or job modifications—can rapidly render a precomputed static schedule infeasible or suboptimal (Aytug et al., 2005).

To manage these uncertainties, the predictive-reactive paradigm has emerged as the dominant framework (Pleier & Schilp, 2024; Vieira et al., 2003). This approach operates in two phases. The first phase generates a predictive baseline schedule using available information. Extensive research has optimized this phase using metaheuristics like Genetic Algorithms (GA) (Kacem et al., 2002; Moghadam et al., 2014), and more recently, DRL-enhanced solvers that adjust evolutionary parameters (Chen et al., 2020; Tang et al., 2024) or leverage Digital Twins for initial planning (Liu et al., 2022).

The second phase, the reactive response, is triggered when a disruption occurs. This phase requires rescheduling actions to repair or revise the baseline schedule, performed either periodically or in an event-driven manner (Vieira et al., 2003). This study focuses on event-driven rescheduling triggered by machine breakdowns. Recent literature (2023–2025) on reactive mechanisms has increasingly adopted data-driven methods. Hybrid frameworks combining DRL with metaheuristics have become standard, where agents adaptively tune parameters (Ma et al., 2025) or select local search operators (Zhang et al., 2024). Furthermore, Digital Twin (DT) technology enhances DRL responsiveness by providing high-fidelity virtual testbeds (Chang et al., 2023; Fang et al., 2023). However, despite these algorithmic advancements, a critical limitation persists: most studies focus on efficiency metrics (e.g., makespan). While some recent works incorporate stability objectives (Hammami et al., 2025), these policies typically operate with a fixed Optimization Scope—either global rescheduling or local rule selection—lacking a systematic mechanism to dynamically adjust the scope based on disruption severity.

Table 1

Summary of representative dynamic scheduling strategies categorized into Global, Partial, and Emerging approaches.

Ref.	Method	Scope	Obj.	Limitation
Wei et al. (2023)	MBO + Game Theory	Global (All Ops)	Eff., Stab.	High complexity; global scope maintained despite stability approx.
Feng et al. (2025)	Adaptive GA	Global (All Ops)	Load, Eff.	Re-optimizes entire population; ignores original schedule stability.
Gui et al. (2023)	DRL (DDPG)	Global Rule Sel.	Mean Tard.	Learns global rules only; lacks capability for targeted local repair.
Yan et al. (2022)	Double-layer Q-L	Global (All Ops)	Prod., Maint.	Computationally too intensive for real-time; replans all resources.
Abumaizar and Svestka (1997)	AOR Heuristic	Affected Ops	Stab., Eff.	Baseline method; optimization potential strictly limited by local view.
Ren and Liu (2024)	D3QN + MachineRank	Dyn. Rule Sel.	Cmax, On-time	Scope limited to rule selection; prone to myopic decisions.
Li et al. (2025)	Dual-level NSGA-II	Affected Ops	Multi-obj.	Uses fixed mechanisms; cannot expand scope dynamically.
Wang et al. (2025)	Knowledge Pred.	Partial Scope	Energy, Cmax	Relies on learned patterns; scope definition is implicit/static.
Ma et al. (2025)	DRL-assisted GA	Global (Param.)	Quality, Conv.	DRL optimizes GA parameters, not the rescheduling scope itself.
Zhang et al. (2024)	DQN Memetic Alg.	Selective Ops	Energy, Cmax	Scope implicitly determined by selected local search operators.
Chang et al. (2023)	DT + DQN	Real-time Disp.	Cmax	Focuses on state tracking; lacks explicit scope control variable.
Hammami et al. (2025)	PPO (DRL)	Event-driven	Eff., Stab.	Addresses stability via rewards, but scope remains implicit.

2.2. Rescheduling strategies

Upon triggering a rescheduling event, the primary decision concerns the strategy selection. Existing methods can be classified along a spectrum defined by their Optimization Scope—the extent to which the existing schedule is modified.

Minimal-intervention strategies occupy one end of the spectrum, prioritizing computational efficiency and stability. The most prominent example is Right-Shift Rescheduling (RSR), which postpones the affected operation and its successors until feasibility is restored (Herrmann, 2006). Due to its negligible computational cost, RSR serves as a standard baseline in comparative studies (Abumaizar & Svestka, 1997). It represents a zero-optimization decision that preserves the original sequence. However, this stability is achieved at the expense of solution quality, often leading to performance degradation.

Conversely, complete rescheduling (CRS), or global rescheduling, treats all unfinished operations as a new static FJSP instance (Cowl-ling & Johansson, 2002). Implementation approaches vary from exact methods like Mixed-Integer Linear Programming (MILP) (Mairhofer et al., 2025) to metaheuristics and game-theory-based algorithms for larger instances (Feng et al., 2025; Wei et al., 2023). Recent learning-based methods include DRL agents for global dispatching (Gui et al., 2023) and complex Double-layer Q-learning architectures (Yan et al., 2022; Zhou et al., 2023). Although CRS theoretically offers the highest potential solution quality, it incurs prohibitive computational costs for real-time applications and often results in severe schedule nervousness due to the disregard for stability (Pujawan, 2004).

Intermediate strategies, known as local or partial rescheduling, aim to balance solution quality, efficiency, and stability. The foundational concept is Affected Operations Rescheduling (AOR), introduced by Abumaizar and Svestka (1997), which confines re-optimization to operations directly or indirectly affected by a disruption. Implementations include dynamic rule selection based on machine ranking (Ren & Liu, 2024), evolutionary algorithms with specific repair mechanisms (Li et al., 2025), and knowledge-driven heuristics using inverse diffusion prediction (Wang et al., 2025).

Finally, emerging research integrates Digital Twins and advanced DRL into what can be termed “black-box” scope strategies. In approaches such as parameter-tuning DRL (Ma et al., 2025), memetic algorithms with selective operators (Zhang et al., 2024), or DT-based dispatching (Chang et al., 2023; Hammami et al., 2025), the Optimization Scope is often implicit—determined by the learning agent or the specific operator selected—rather than explicitly defined. Table 1 summarizes these representative studies, categorizing them by methodological approach and Optimization Scope.

2.3. Optimization scope gaps

A review of the literature indicates that the predominant focus has been on “how to reschedule”—developing more powerful solvers for a pre-selected scope. In contrast, the strategic question of “how much to

reschedule”—selecting the optimization scope itself—remains largely unaddressed. Three critical research gaps are identified:

- 1. Absence of a Formal Framework for Optimization Scope Analysis.** While the literature implicitly acknowledges various scopes (e.g., repair, partial, complete) (Vieira et al., 2003), a formal, quantifiable definition of the *Optimization Scope* is lacking. Consequently, there is no systematic analysis of the trade-offs among solution quality, computational cost, and stability as the scope is incrementally expanded.
- 2. Unexplored Failure Modes of Myopic Local Strategies.** The spectrum between minimal scope (RSR) and broader AOR scope includes finer-grained strategies, such as greedy, single-operation-focused heuristics. Despite their simplicity, the potential adverse effects of these myopic strategies—referred to in this paper as the “Greedy Trap”—have not been formally characterized.
- 3. Scarcity of Practically Balanced Rescheduling Strategies.** Current research tends to be polarized, either pursuing theoretical optimality via CRS at the cost of real-time viability, or defaulting to simplistic strategies like RSR which sacrifice performance for sequence stability. There is a clear need for a strategy that co-optimizes quality, efficiency, and stability, suitable for deployment in production-grade MES.

This paper addresses these gaps by: (1) introducing a formal framework for Optimization Scope; (2) characterizing the “Greedy Trap” as a design principle; and (3) developing ERRE, a balanced and computationally efficient rescheduling strategy.

3. Optimization scope framework and rescheduling strategies

This section details the mathematical modeling framework used to analyze the impact of the Optimization Scope on rescheduling performance. We first present the formulation of the static Flexible Job Shop Scheduling Problem (FJSP), followed by the definition of the dynamic rescheduling scenario triggered by machine breakdowns. Finally, we define the optimization objectives used to evaluate strategy performance.

3.1. Problem description

The key notations used throughout the mathematical modeling are summarized in Table 2.

3.1.1. Mathematical formulation of static FJSP

The static FJSP is defined by the following components:

- **Jobs:** A set of n independent jobs, $J = \{J_1, J_2, \dots, J_n\}$.

Table 2
Summary of notations.

Symbol	Description
n, m	Total number of jobs and machines
n_i	Number of operations of job J_i
J, M, O	Sets of jobs, machines, and operations
O_{ij}	The j th operation of job J_i
O_U	Set of unstarted operations at T_{fail}
O_P	Set of in-process operations at T_{fail}
M_{ij}	Set of eligible machines for operation O_{ij}
Ω	Optimization Scope (subset of unstarted operations)
p_{ijk}	Processing time of O_{ij} on machine M_k
S_{orig}, S_{new}	Original schedule and rescheduled schedule
T_{fail}, T_{repair}	Time of machine breakdown and repair
M_{fail}	The machine encountering a breakdown
ST_{ij}	Start time of operation O_{ij}
CT_{ij}	Completion time of operation O_{ij}
MA_{ij}	Assigned machine for operation O_{ij}
C_{max}	Makespan of the schedule
f_1, f_2	Objectives for quality (makespan) and stability (ADST)

- **Operations:** Each job J_i consists of a sequence of operations $O_i = \{O_{i1}, O_{i2}, \dots, O_{in_i}\}$, subject to precedence constraints. O_{ij} denotes the j th operation of job J_i , and $O = \bigcup_{i=1}^n O_i$ represents the set of all operations.
- **Machines:** A set of m distinct machines, $M = \{M_1, M_2, \dots, M_m\}$.
- **Processing Flexibility:** Each operation O_{ij} can be processed on a subset of eligible machines $M_{ij} \subseteq M$.
- **Processing Time:** The processing time of O_{ij} on machine $M_k \in M_{ij}$ is deterministic, denoted by p_{ijk} .

A feasible schedule S assigns a machine $MA_{ij} \in M_{ij}$ and a start time ST_{ij} to each operation, satisfying:

1. **Precedence Constraints:** An operation must not start before its predecessor completes:

$$ST_{i,j+1} \geq ST_{ij} + p_{i,j,MA_{ij}}, \quad \text{for } j = 1, \dots, n_i - 1. \quad (1)$$

2. **Capacity Constraints:** A machine can process at most one operation at a time.

The initial predictive schedule, S_{orig} , is generated to minimize the makespan:

$$C_{max}(S_{orig}) = \max_{O_{ij} \in O} \{ST_{ij}^{orig} + p_{i,j,MA_{ij}^{orig}}\}. \quad (2)$$

3.1.2. Dynamic rescheduling scenario

This study addresses rescheduling triggered by unexpected machine breakdowns. A breakdown on machine M_{fail} defines an unavailability window $[T_{fail}, T_{repair}]$. Consistent with standard predictive-reactive approaches, we assume the estimated repair duration is known at the disruption epoch T_{fail} to allow for future planning. To ensure robustness, repair durations are generated stochastically in the experimental validation (Section 4.2).

Operations are handled based on their status at T_{fail} . Operations interrupted by the breakdown ($O_{ij} \in O_P$) follow a preemptive-repeat policy (Peng et al., 2025; Yi et al., 2025): they are aborted and returned to the unstarted set (O_U) for rescheduling. Conversely, operations currently processing on healthy machines continue undisturbed.

Fig. 1 illustrates this process: panel (a) captures the interruption of an active operation by a breakdown (red dashed box), while panel (b) depicts the resulting schedule S_{new} . In this rescheduled state, the interrupted operation is reassigned to an alternative resource, while operations on non-faulty machines continue across the T_{fail} line without interruption, serving as fixed hard constraints. Consequently, this reallocation may induce right-shifts in subsequent operations due to resource contention.

3.1.3. Rescheduling objectives

The rescheduling problem requires balancing three conflicting objectives: solution quality, schedule stability, and computational efficiency.

1. **Solution Quality (Makespan):** To maintain production efficiency, the primary objective is to minimize the makespan of the new schedule (S_{new}):

$$f_1 = \min C_{max}(S_{new}) = \min_{O_{ij} \in O_U \cup O_P} \max \{CT_{ij}^{new}\}, \quad (3)$$

where CT_{ij}^{new} is the completion time of operation O_{ij} .

2. **Schedule Stability (Temporal Deviation):** To minimize operational disruption, we quantify stability using the *sum of absolute deviations in start times* (ADST) for all unstarted operations (Kulcsár et al., 2025):

$$f_2 = \min \sum_{O_{ij} \in O_U} |ST_{ij}^{new} - ST_{ij}^{orig}|. \quad (4)$$

This metric (f_2) explicitly targets *temporal stability*. It is important to note that this metric specifically quantifies *temporal stability*. While some strategies may preserve the machine sequence (sequence stability), they may still incur high ADST values if they cause significant delays to downstream operations. A lower ADST value corresponds to reduced nervousness on the shop floor.

3. **Computational Efficiency:** The responsiveness of the strategy is measured by the CPU time (T_{cpu}). While not part of the explicit cost function, minimizing T_{cpu} is a constraint for meeting the real-time requirements of MES integration, distinguishing practical online strategies from computationally intensive offline methods.

3.2. Hierarchical definition of optimization scope

To systematically quantify the rescheduling magnitude, we define the *Optimization Scope*, denoted by Ω . Ω represents the subset of unstarted operations ($\Omega \subseteq O_U$) eligible for re-optimization regarding machine assignment and sequencing. Operations outside this scope ($O_{ij} \in O_U \setminus \Omega$) retain their original machine assignments and relative sequences, with only their start times subject to passive shifting to maintain feasibility.

We categorize the Optimization Scope into four hierarchical levels, ranging from minimal intervention to global re-optimization.

3.2.1. Level 1: Zero optimization scope (Ω_0)

The most constrained level, Zero Optimization Scope ($\Omega_0 = \emptyset$), represents a purely reactive repair mechanism serving as a baseline. The representative implementation is Pure Right-Shift Rescheduling (Pure-RSR). In this deterministic approach, the start times of affected operations are shifted backward to satisfy constraints, while preserving all original machine assignments ($MA_{ij}^{new} = MA_{ij}^{orig}$) and sequences. This guarantees maximal *sequence stability* as no machine changes occur. However, regarding *temporal stability* (f_2), RSR can be surprisingly unstable: by passively propagating delays to all successor operations, it often results in large deviations in start times (high ADST), typically at the expense of solution quality (f_1).

3.2.2. Level 2: Greedy optimization scope (Ω_G)

This level introduces localized optimization by defining the scope Ω_G to include only immediate schedulable operations at any decision point. A representative algorithm is Active & Aggressive Right-Shift Rescheduling (A²-RSR). A²-RSR operates as an event-driven constructive heuristic. Upon disruption, it iteratively constructs the schedule by selecting a high-priority operation O_{ij}^* (e.g., based on Earliest Start Time) and assigning it to the machine yielding the earliest completion time. This approach is “greedy” as it optimizes locally without evaluating the downstream impact on the global schedule.

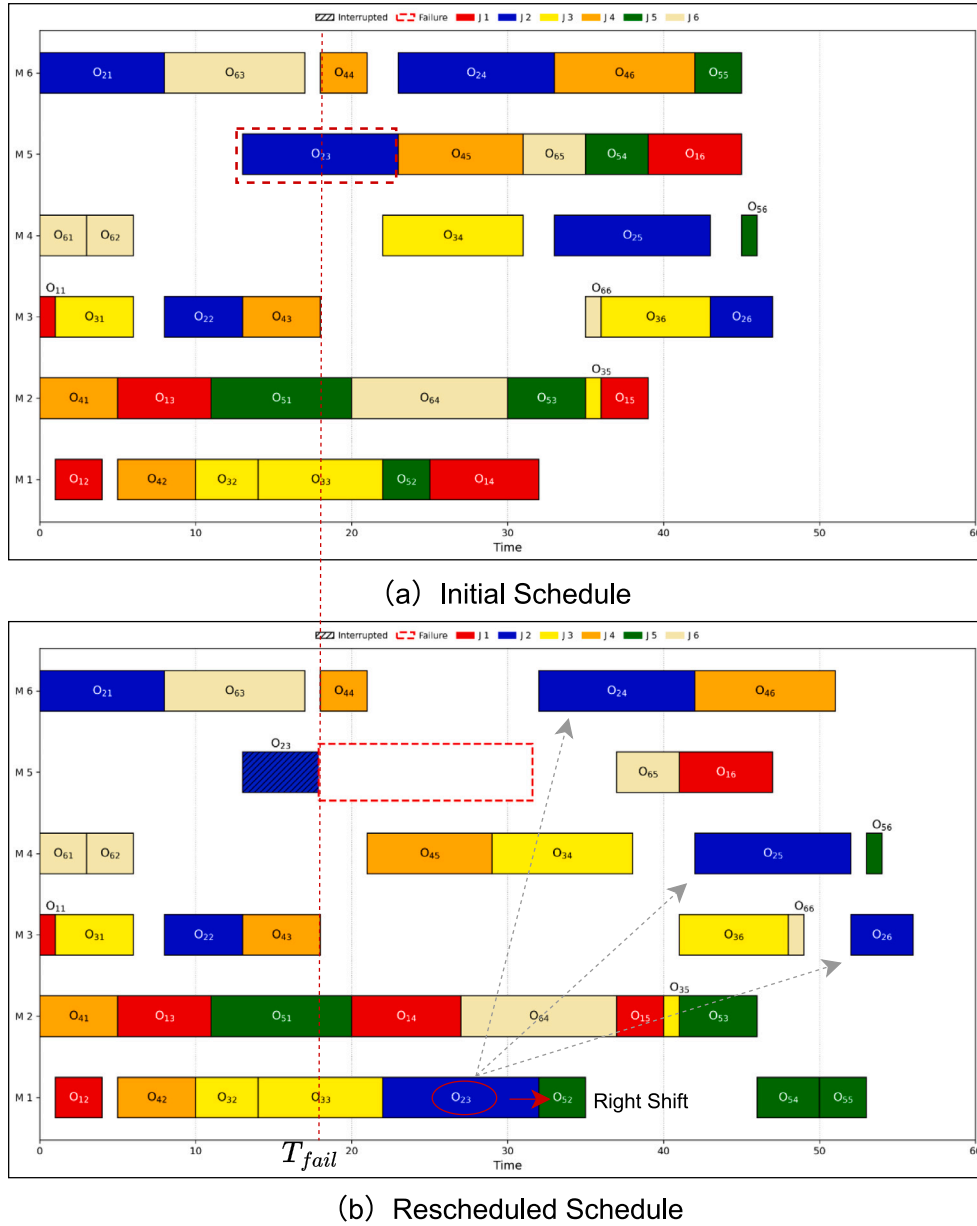


Fig. 1. Schematic of the rescheduling logic under machine breakdown: (a) Initial schedule interrupted by a breakdown; (b) Rescheduled schedule where the interrupted task is reallocated, while healthy operations remain fixed constraints.

3.2.3. Level 3: Affected scope (Ω_A)

The third level defines the *Affected Scope* (Ω_A), comprising operations whose planned execution is rendered potentially suboptimal by the disruption. This scope aims to isolate the critical subproblem requiring re-optimization, thereby avoiding unnecessary computation on unaffected schedule segments.

To operationalize Ω_A , we propose the ERRE (Event-driven Rescheduling with Elite Evolution) framework. ERRE consists of two modules: problem identification and solution evolution. The workflow is illustrated in Fig. 2.

ASIA: Affected-scope identification algorithm. The first module, ASIA, is responsible for precisely filtering the set of operations that require rescheduling. As detailed in Algorithm 1, ASIA employs a Breadth-First Search (BFS) strategy to trace the “ripple effect” of the disruption. Crucially, the algorithm distinguishes between flexible operations and those strictly constrained by current execution states (e.g., operations already running on healthy machines, denoted as O_{fixed}). By explicitly

excluding O_{fixed} from the scope, ASIA ensures that the rescheduling process does not disrupt stable, ongoing production activities. The logic proceeds in three steps:

1. **Initialization (Seeds):** The process begins by identifying directly affected operations—specifically, those interrupted by the breakdown or blocked by the repair window on the failed machine—excluding any operations currently running on healthy machines (O_{fixed}). These tasks form the initial seed set of Ω_A .
2. **Propagation:** The scope is then iteratively expanded using a BFS approach. For every operation added to Ω_A , the algorithm appends its downstream dependencies: (1) *Job Successors* (all subsequent operations of the same job), and (2) *Machine Successors* (the next valid, unstated operation scheduled on the same machine in S_{orig} , skipping any completed tasks).
3. **Termination:** The propagation continues until the “ripple effect” naturally dissipates or reaches the boundary of the fixed

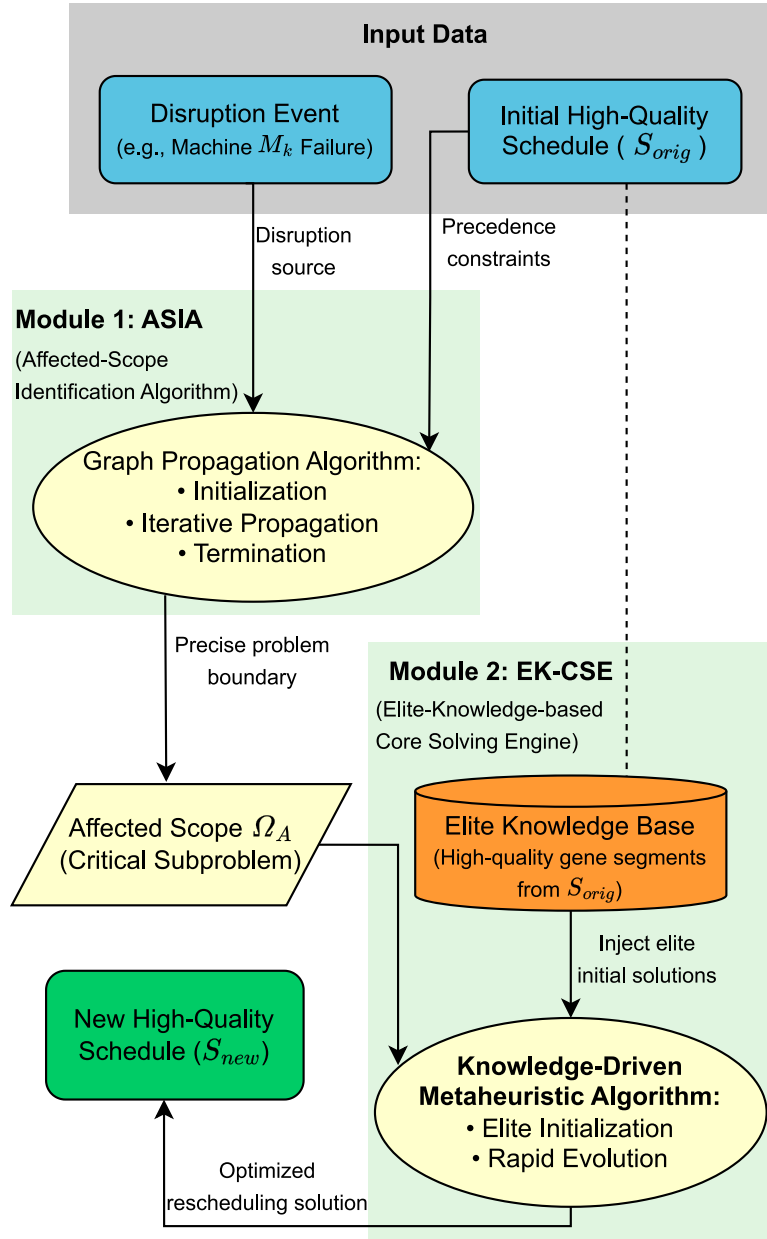


Fig. 2. The computational workflow of the proposed ERRE framework.

schedule, ensuring Ω_A captures the complete chain of impact without over-expanding.

EK-CSE: Elite-knowledge-based core solving engine. Once Ω_A is defined, ERRE invokes the EK-CSE module, a knowledge-driven metaheuristic implemented as a lightweight Memetic Algorithm. EK-CSE re-optimizes machine assignments and sequencing strictly within Ω_A . A key feature is “Knowledge Inheritance”, where the initial population is seeded with solution fragments from S_{orig} rather than random initialization. This enables rapid convergence by leveraging the pre-existing high-quality plan structure.

3.2.4. Level 4: Global future scope (Ω_F)

The final level, *Global Future Scope* ($\Omega_F = O_U$), encompasses all unstarted operations. The corresponding strategy, Complete Rescheduling (CRS), reformulates the remaining problem as a new static FJSP instance. CRS employs global optimization algorithms to regenerate machine assignments and sequences from scratch. While CRS provides

a theoretical reference for solution quality (f_1), it is often constrained by real-time computational limits. Regarding stability, CRS presents a complex profile: while it completely disrupts the original machine assignments (low sequence stability), its ability to find tight schedules can effectively compress delays, potentially resulting in better temporal stability (f_2) than passive strategies, albeit at the price of complete structural reconfiguration.

4. Computational experiments

This section details the experimental framework designed to evaluate the performance of rescheduling strategies across the four optimization scopes. All algorithms were implemented in Python 3.7 and executed on a workstation with an Intel Core i7-9700 CPU and 32 GB of RAM. The experiments aim to compare these strategies regarding solution quality, schedule stability, and computational efficiency within a controlled simulation environment. The following subsections outline the benchmark instances, dynamic disruption scenarios, algorithm implementation details, and performance metrics.

Algorithm 1 Affected-Scope Identification Algorithm (ASIA)

Require: Initial Schedule S_{orig} , Disruption Info (T_{now} , M_{fail} , T_{repair}), Set of Unstarted Operations O_U , Set of Fixed Operations O_{fixed}

Ensure: Affected Scope Ω_A

- 1: Initialize $\Omega_A \leftarrow \emptyset$, Queue $Q \leftarrow \emptyset$
- 2: // **Step 1: Identify Initial Seeds (Directly Affected)**
- 3: **for** each operation $O_{ij} \in O_U$ **do**
- 4: $is_interrupted \leftarrow O_{ij}$ was running on M_{fail} at T_{now}
- 5: $is_blocked \leftarrow S_{orig}$ assigned O_{ij} to M_{fail} within $[T_{now}, T_{repair}]$
- 6: **if** ($is_interrupted$ **or** $is_blocked$) **and** $O_{ij} \notin O_{fixed}$ **then**
- 7: $\Omega_A \leftarrow \Omega_A \cup \{O_{ij}\}$
- 8: $Q.enqueue(O_{ij})$
- 9: **end if**
- 10: **end for**
- 11: // **Step 2: Iterative Propagation (BFS)**
- 12: **while** Q is not empty **do**
- 13: $O_{curr} \leftarrow Q.dequeue()$
- 14: // (a) **Job Successor Propagation**
- 15: **for** each subsequent operation O_{next_job} of O_{curr} **do**
- 16: **if** $O_{next_job} \notin \Omega_A$ **and** $O_{next_job} \notin O_{fixed}$ **then**
- 17: $\Omega_A \leftarrow \Omega_A \cup \{O_{next_job}\}$
- 18: $Q.enqueue(O_{next_job})$
- 19: **end if**
- 20: **end for**
- 21: // (b) **Machine Successor Propagation**
- 22: Find the first successor O_{next_mach} of O_{curr} on S_{orig} machine
- 23: **while** O_{next_mach} exists **and** $O_{next_mach} \notin O_U$ **do**
- 24: $O_{next_mach} \leftarrow$ next successor on machine sequence
- 25: **end while**
- 26: **if** O_{next_mach} exists **and** $O_{next_mach} \notin \Omega_A$ **and** $O_{next_mach} \notin O_{fixed}$ **then**
- 27: $\Omega_A \leftarrow \Omega_A \cup \{O_{next_mach}\}$
- 28: $Q.enqueue(O_{next_mach})$
- 29: **end if**
- 30: **end while**
- 31: **return** Ω_A

Table 3
Characteristics of the selected benchmark instances.

Instance	Jobs	Machines	Operations	Origin
MT06	6	6	36	Hurink et al. (vdata)
MK01	10	6	55	Brandimarte
MK04	15	8	90	Brandimarte
MT10	10	10	100	Hurink et al. (vdata)
MK06	10	15	150	Brandimarte
LA38	15	15	225	Hurink et al. (vdata)
MK10	20	15	240	Brandimarte
LAR03	50	60	250	Behnke & Geiger
LA31	30	10	300	Hurink et al. (vdata)
LAR04	100	60	500	Behnke & Geiger

4.1. Test instances

We selected a diverse set of standard benchmark instances widely used in FJSP research. These include small- to medium-scale sets from Hurink et al. (1994) and Brandimarte (1993), as well as large-scale, high-flexibility instances from Behnke and Geiger (2012). To ensure a comprehensive benchmarking scope, the selection covers three distinct scales: Small (e.g., MT06, MK01), Medium (e.g., MK10, LA38), and Large (e.g., LAR04, 500 operations). This diversity allows for a rigorous evaluation of strategy scalability and robustness across different problem complexities. While representative results are discussed in Section 5, the complete experimental data for all tested instances is provided in Appendix C to ensure full benchmarking transparency. Table 3 summarizes the characteristics of the selected instances. In addition to

Table 4

Parameter configurations for the seven dynamic disruption scenarios.

ID	Scenario	N_f	Int.	Tgt.	Logic
S1	Single-Light	1	$U[0.5, 1.5]$	Med./Low	Late phase
S2	Single-Medium	1	$U[2.0, 4.0]$	Med.	Mid phase
S3	Single-Heavy	1	$U[5.0, 8.0]$	High	Early phase
S4	Concurrent-Adj.	2	$U[2.0, 4.0]$	Any 2	Adj. windows
S5	Concurrent-Overlap	2	$U[5.0, 8.0]$	2 High	Overlap windows
S6	Sequential-Medium	2	$U[2.0, 4.0]$	Any 2	Seq. (early & late)
S7	Avalanche-Chaos	3	$U[9.0, 15.0]$	Any 3	High overlap

Table 5

Detailed parameter settings and operator definitions to ensure reproducibility.

Parameter	ERRE/ERRE-Full	CRS
Scope	Ω_A/Ω_F	Ω_F
Init. Strategy	Elite Expansion	Hybrid (G/L/R 6:3:1)
Pop. Size (N_p)	50	50
Max. Gen.	30	50
Selection	Truncation	Tourn. ($k = 3$)
Crossover Op.	N/A (Single-parent)	POX (OS)/2-Point (MA)
Mutation Op.	Swap/Uniform ($P_m = 0.2$)	Swap/Uniform ($P_m = 0.2$)
Elitism Ratio	10%	10%

Note: ERRE-Full shares identical parameters with ERRE. G/L/R: Global/Local/Random.

these standard benchmarks, we also constructed a specialized synthetic instance named *Decoupled* 10×10 to perform a targeted ablation study on structural stability, which will be detailed in Section 5.4.

4.2. Dynamic scenario generation

To evaluate robustness, we developed a scenario generator that produces customized machine breakdown events based on instance properties. The generator operates on three principles: (1) disruption intensity (T_{dur}) is normalized by the average operation processing time ($\bar{p}$); (2) target machines are selected based on workload (high, medium, or low load); and (3) disruption timing is controlled to occur at specific production phases (early, middle, or late).

Based on these principles, seven dynamic scenarios (S1–S7) were designed to simulate varying degrees of system stress:

- **Single-Machine Disruptions (S1–S3):** Assess responsiveness to isolated failures. S1 represents a light, late-phase disruption; S2 a medium, mid-phase disruption; and S3 a heavy, early-phase disruption on a bottleneck machine.
- **Multi-Machine Concurrent Disruptions (S4–S5):** Test management of cascading events. S4 simulates a “chain reaction” with two adjacent breakdowns, while S5 models a critical incident with two overlapping, high-intensity breakdowns on high-load machines.
- **Multi-Machine Sequential Disruptions (S6):** Evaluate resilience to repeated shocks through two non-overlapping breakdowns (early and late).
- **Systemic Failure Simulation (S7):** The *Avalanche-Chaos* scenario simulates a near-system-collapse event with three concurrent, high-intensity breakdowns.

Table 4 details the configurations. To ensure statistical reliability, each instance-scenario pair was replicated 10 times using fixed, deterministically generated random seeds (Base Seed = 2025). Reported results are averages over these independent runs. For the ablation study in Section 5.4, we utilized a specific subset of these scenarios tailored to the *Decoupled* 10×10 instance to isolate the impact of optimization scope.

4.3. Comparison algorithms and parameter settings

We compared four algorithms corresponding to the hierarchical optimization scopes: Pure-RSR (Ω_0), A^2 -RSR (Ω_G), ERRE (Ω_A), and CRS

(Ω_F). Pure-RSR and A²-RSR are deterministic heuristics. ERRE and CRS are evolutionary algorithms with distinct architectures tailored to their scopes.

Both evolutionary algorithms utilize the same two-vector encoding (Operation Sequence and Machine Assignment) (Li & Gao, 2016) and context-aware active decoding. The evolutionary process terminates upon reaching a pre-defined maximum number of generations. To ensure the reproducibility of the experiments, we explicitly define the genetic operators used in both CRS and the evolutionary component of ERRE. For the Operation Sequence (OS) vector, Precedence Operation Crossover (POX) is employed to strictly preserve job precedence constraints, complemented by Swap Mutation for local perturbation. For the Machine Assignment (MA) vector, Two-Point Crossover and Uniform Mutation are applied to explore alternative resource allocations. The specific parameter settings, chosen based on the sensitivity analysis in Appendix B, are detailed in Table 5.

- **CRS (Global Optimizer):** CRS operates as a standard Genetic Algorithm exploring the global space (Ω_F). It uses hybrid initialization, tournament selection ($k = 3$), and mixed crossover. To ensure the reliability of this baseline, we validated the algorithm's performance on standard static benchmark instances; the results, provided in Appendix A, confirm its capability to find optimal or near-optimal solutions (average gap < 5% relative to BKS). While static scheduling often uses large populations, we set $N_p = 50$. Our sensitivity analysis (see Appendix B) indicates that increasing N_p to 500 yields less than 10% quality improvement but incurs a 40-fold increase in CPU time, violating real-time constraints. Thus, the configuration ($N_p = 50, Gen = 50$) balances quality and speed. Note that CRS is allocated a higher generation limit (50) compared to ERRE (30) to compensate for its larger search space, ensuring it has ample opportunity to converge within the real-time feasibility window.
- **ERRE (Local Refiner):** ERRE is a lightweight Memetic Algorithm for the partial scope (Ω_A). To ensure a rigorous comparison, ERRE utilizes the same population size ($N_p = 50$) and elitism ratio (10%) as CRS, but employs a fundamentally different initialization logic:
 1. **Initialization (Elite Expansion):** Instead of random generation, ERRE inherits elite seeds from the pre-disruption population. These seeds are first repaired to satisfy the new constraints and then **cloned** to fill the target population size ($N_p = 50$). This creates a high-quality initial population focused on the promising region of the search space.
 2. **Evolution:** ERRE omits crossover (Single-parent evolution) to preserve the structure of inherited elites, relying on truncation selection and mutation ($P_m = 0.2$).
 3. **Local Search:** A critical-path-based local search accelerates convergence within the limited 30 generations.

4.4. Performance metrics

We evaluated performance across three dimensions:

(1). **Solution Quality:** Production efficiency is measured by the primary objective f_1 (Makespan), normalized as the **Relative Percentage Deviation (RPD)**. For an algorithm A_i , RPD is defined as:

$$RPD_{A_i}(\%) = \frac{C_{A_i} - C_{best}}{C_{best}} \times 100\% \quad (5)$$

where C_{best} is the best solution found among all algorithms. We report the mean RPD (**M-RPD**) and standard deviation (**RSD-RPD**) to indicate quality and stability, respectively.

(2). **Schedule Stability:** Stability is quantified using the **ADST** (Absolute Deviation of Start Times, Eq. (4)). Lower ADST values indicate reduced shop-floor nervousness.

(3). **Computational Efficiency:** Efficiency is measured by the average **CPU Time** (seconds) required to generate a feasible reschedule.

5. Results and analysis

This section reports the results of the computational experiments evaluating the impact of optimization scope on rescheduling performance. Four strategies representing distinct optimization scopes—Pure-RSR, A²-RSR, ERRE, and CRS—were tested across a range of benchmark instances and dynamic disruption scenarios. In addition, a targeted ablation study was conducted to isolate the specific contribution of the scope definition mechanism. The following analysis compares these strategies to identify performance trade-offs and validate the proposed hypotheses.

5.1. Overall performance overview

We begin by analyzing the aggregated performance of the four strategies across all 70 test scenarios (10 instances $\times$ 7 scenarios). Fig. 3 presents the key statistical metrics: mean Relative Percentage Deviation (M-RPD), standard deviation of RPD (RSD-RPD), and the success count for finding the best solution (#Best). These metrics provide a macroscopic view of average quality and quality stability (robustness). Additionally, Table 6 details the performance on four representative instances to illustrate specific behavioral patterns, while the detailed results for the remaining instances are provided in Appendix C to validate the consistency of these findings.

As shown in Fig. 3, the aggregated data establishes a clear performance hierarchy. Under the standardized experimental setting ($N_p = 50$), the ERRE strategy demonstrates a significant advantage. It achieves an M-RPD of 0.76% (left panel), compared to 4.15% for the global baseline CRS. This performance gap supports the hypothesis that, given identical computational resources, concentrating optimization on a precisely defined sub-problem (Ω_A) is more effective than dispersing it across the global search space. Furthermore, the right panel of Fig. 3 indicates that ERRE secures the best solution in 52 out of 70 scenarios, a substantial margin over CRS (13 times) and A²-RSR (5 times). Both myopic strategies, Pure-RSR and A²-RSR, exhibit M-RPD values exceeding 13%, highlighting their limitations for high-quality scheduling.

The detailed breakdown in Table 6 confirms ERRE's dominance over the global baseline, particularly in complex scenarios. On medium-to-large scale instances such as MT10 and LAR04, ERRE consistently outperforms other methods. For example, in MT10, ERRE achieves the best Makespan in all 7 scenarios; in S1, it reduces the Makespan to 689.35, compared to 728.62 for CRS. Similarly, on the large-scale LAR04, ERRE surpasses CRS in all reported scenarios. This suggests that for complex dynamic problems, the global search space of CRS is too vast to be explored effectively within the strict time constraints (50 generations). Conversely, ERRE leverages initial elite knowledge and focuses on the Affected Scope, enabling deeper exploitation of high-quality solutions.

Regarding stability, the limitations of greedy decision-making become apparent. A²-RSR exhibits a high RSD-RPD of 12.38%, indicating volatility across different problem structures. While A²-RSR performs adequately on the flexible LAR04 instance, it degrades on the tightly-coupled MK10. In the MK10-S3 scenario, its New Cmax (417.14) is worse than the passive Pure-RSR (302.72) and significantly inferior to ERRE (274.52). This underscores ERRE's ability to maintain solution quality consistently, avoiding the "Greedy Trap" that limits myopic heuristics.

Finally, regarding computational efficiency, ERRE shows a distinct advantage over the global baseline. Although ERRE involves an

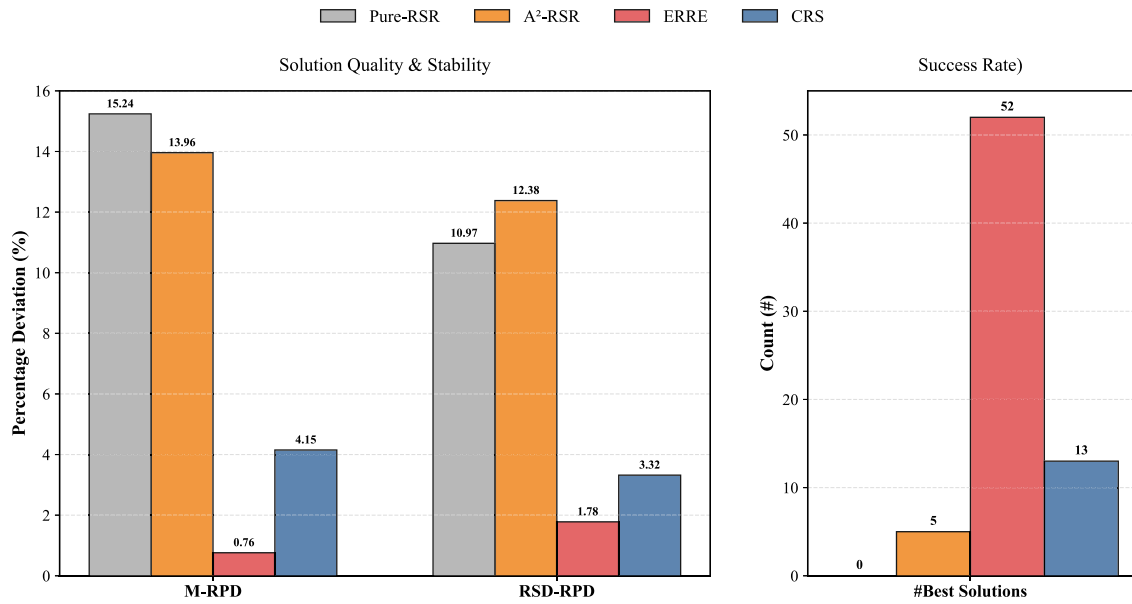


Fig. 3. Aggregated performance comparison of the four strategies across all 70 scenarios. The left panel shows solution quality and stability metrics (M-RPD and RSD-RPD, lower is better), while the right panel displays the success rate in finding the best solution (#Best, higher is better).

Table 6

Performance comparison of four strategies on selected instances under dynamic scenarios.

Ins.	Sce.	Metrics (New Gmax/ADST/CPU Time (s))			
		Pure-RSR	A ² -RSR	ERRE	CRS
MT06	S1	55.07/1.36/< 0.01	55.08/0.80/< 0.01	49.94/0.20/0.20	51.94/0.92/0.73
	S2	65.57/6.83/< 0.01	63.04/3.72/< 0.01	51.86/2.21/0.24	54.36/3.04/0.81
	S3	83.64/19.43/< 0.01	61.64/7.91/< 0.01	54.05/4.72/0.30	57.99/5.83/0.87
	S4	70.62/9.19/< 0.01	69.14/6.18/< 0.01	55.46/2.99/0.51	59.39/5.84/1.67
	S5	85.78/21.63/< 0.01	64.90/7.59/< 0.01	61.50/8.66/0.60	60.17/7.99/1.73
	S6	72.62/12.12/< 0.01	64.43/6.85/< 0.01	58.21/4.04/0.50	59.29/6.08/1.63
	S7	129.37/41.05/< 0.01	102.20/14.37/< 0.01	105.29/18.20/0.82	101.81/25.05/2.63
MT10	S1	742.67/9.58/< 0.01	772.65/10.29/< 0.01	689.35/2.18/0.56	728.62/7.69/2.17
	S2	840.52/44.77/< 0.01	844.63/37.16/< 0.01	748.61/25.28/0.87	800.46/44.07/2.54
	S3	992.22/184.86/< 0.01	967.60/109.41/0.01	781.51/51.80/1.02	877.52/98.50/2.76
	S4	868.30/83.99/< 0.01	905.86/53.87/0.01	764.01/29.34/1.69	828.13/59.73/5.04
	S5	1031.03/243.90/< 0.01	945.60/119.66/0.01	876.86/92.04/2.08	914.58/132.98/5.58
	S6	950.05/128.26/< 0.01	984.94/101.38/0.01	793.15/48.38/1.74	898.20/111.55/5.29
	S7	1430.72/441.86/< 0.01	1172.59/165.22/0.01	1129.13/164.26/2.91	1179.32/243.32/8.30
MK10	S1	244.15/1.32/< 0.01	272.97/4.51/0.01	243.44/1.72/1.42	243.01/3.40/8.25
	S2	267.88/10.42/< 0.01	341.21/17.88/0.01	257.28/7.86/2.14	256.83/13.26/9.33
	S3	302.72/45.22/< 0.01	417.14/61.37/0.01	274.52/23.93/2.80	278.84/26.80/10.54
	S4	276.17/16.31/0.01	361.06/24.78/0.02	265.71/11.70/4.44	264.88/12.24/18.31
	S5	311.41/51.69/0.01	400.84/49.49/0.02	280.85/29.03/4.85	283.93/28.92/18.68
	S6	305.42/35.87/< 0.01	415.95/57.31/0.02	270.72/17.96/4.62	273.18/24.85/18.52
	S7	386.72/105.69/< 0.01	443.01/64.23/0.03	333.21/53.20/6.63	345.36/61.51/28.11
LAR04	S1	434.90/1.35/< 0.01	433.08/2.56/0.02	417.70/1.23/3.31	433.33/3.50/30.35
	S2	464.24/5.13/< 0.01	429.44/19.19/0.06	419.90/3.47/4.90	448.81/25.08/33.78
	S3	545.22/29.76/< 0.01	437.09/65.57/0.13	444.50/12.30/6.09	481.71/85.20/35.58
	S4	475.93/11.43/0.01	436.70/15.98/0.10	426.10/6.11/9.67	457.20/26.53/67.88
	S5	523.37/34.32/0.01	448.09/67.28/0.24	445.70/15.86/9.84	489.15/91.13/57.96
	S6	490.13/21.54/0.01	435.56/66.91/0.16	436.74/10.89/9.77	466.80/78.04/64.90
	S7	682.94/87.67/0.01	476.10/64.56/0.26	492.95/24.00/19.67	528.86/92.47/87.29

Note: Bold values indicate the best result among the four strategies.

evolutionary process, the reduced problem size accelerates convergence. As indicated in Table 6, for the computationally demanding scenario LAR04-S7, CRS requires 87.29 s, a latency potentially disruptive to production. In contrast, ERRE completes rescheduling in 19.67 s—approximately 4.5 times faster—while delivering a superior Makespan (492.95 vs 528.86). This represents a favorable trade-off: ERRE achieves high solution quality with computational costs suitable for online industrial deployment, effectively bridging the gap between heuristic speed and metaheuristic quality.

In summary, the analysis identifies ERRE as the superior strategy. By strategically defining the Optimization Scope, it addresses the inefficiency of global search baselines and the myopia of greedy heuristics, offering a balanced combination of quality, stability, and speed.

5.2. Analysis of the “greedy trap” phenomenon

The preceding results indicated performance volatility in the greedy strategy, A²-RSR. This section analyzes the failure mode termed the “Greedy Trap”, where a sequence of locally optimal decisions leads to

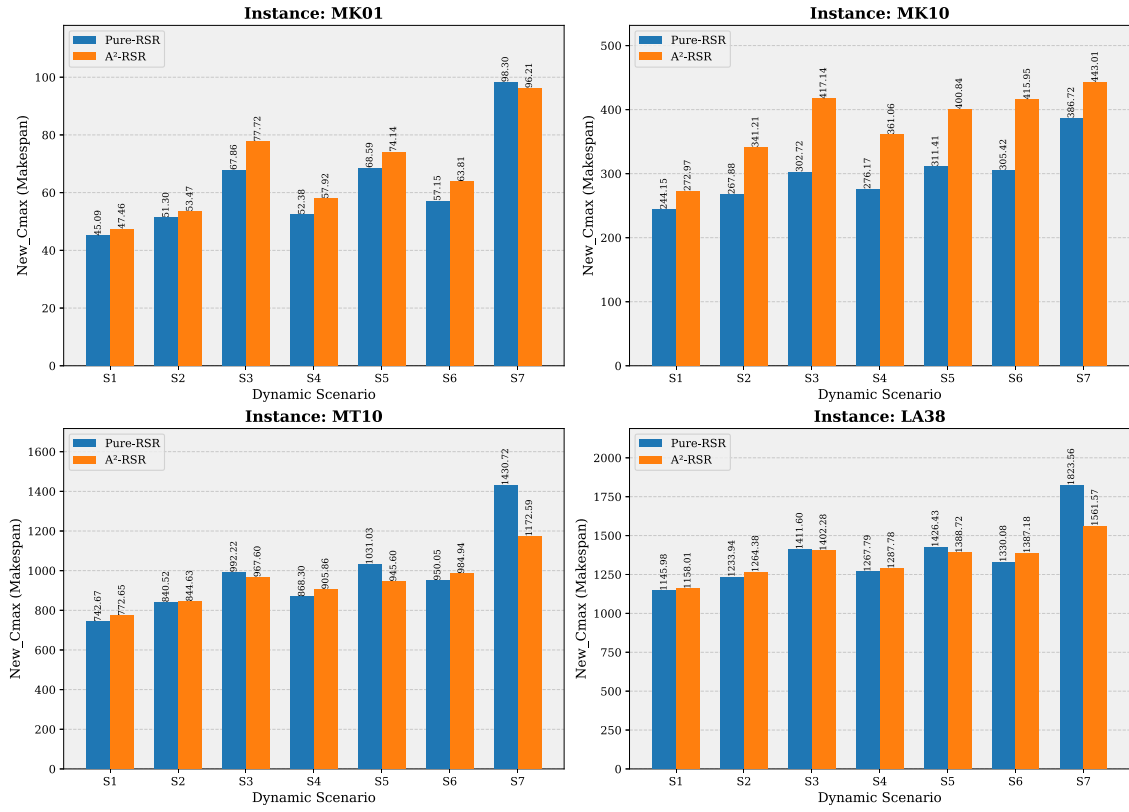


Fig. 4. Makespan comparison between Pure-RSR and A²-RSR, illustrating the “Greedy Trap” phenomenon on representative instances.

global inefficiency. This occurs when seizing a resource for immediate gain triggers cascading delays for subsequent operations.

Fig. 4 compares the makespan of A²-RSR against the passive Pure-RSR. The phenomenon is most evident on the tightly-coupled MK10 instance, where A²-RSR performs worse than Pure-RSR in all seven scenarios. In scenario S3, the makespan of A²-RSR (417.14) is 37.8% higher than that of Pure-RSR (302.72). This suggests that for problems with high structural coupling, myopic active decisions can be detrimental. Similarly, A²-RSR underperforms in four out of seven scenarios on both MT10 and MK01 instances.

However, the “Greedy Trap” is not universal. On the LA38 instance, A²-RSR outperforms Pure-RSR in three scenarios (S3, S5, S7). This indicates that the trap’s activation depends on the interaction between the problem’s topology (e.g., resource contention density) and the disruption characteristics.

The root cause of the “Greedy Trap” is visually conceptualized in Fig. 5. As illustrated, the trap is triggered when a myopic strategy (A²-RSR) prioritizes the earliest completion of a single disrupted task (Job A) without considering resource contention or processing efficiency. In the example, although M1 is the faster machine, the greedy strategy seizes the currently available but slower machine M2. This decision inflicts a double penalty: it not only extends the processing time of Job A (from 2t to 4t) but also inadvertently blocks the critical path for the subsequent Job B, creating a “domino effect” of delays that extends the makespan to 7t. In contrast, a coordinated strategy like ERRE foresees this conflict. It accepts a local sacrifice—waiting for the efficient machine M1 to be repaired—to preserve M2 for Job B, thereby optimizing the global flow (6t).

The mechanism underlying this failure is the myopic nature of the A²-RSR heuristic. By selecting the earliest completion time for a single operation at each step, the algorithm may commit a critical resource, constraining the future solution space. Each greedy decision acts as an irreversible pruning of the search tree. Without a backtracking mechanism, the algorithm is forced down a suboptimal path. The poor

performance on MK10 exemplifies this: frequent assignments to bottleneck machines, while locally optimal, create conflicts for dependent tasks.

In contrast, the passive Pure-RSR strategy preserves the structure of the initial high-quality schedule, avoiding active missteps. This analysis highlights that for complex combinatorial problems like FJSP, employing a strategy with a coordinated perspective, such as ERRE, is essential to prevent decision failures like the “Greedy Trap”.

5.3. Evaluation of ERRE: Robustness and efficiency

The preceding analysis highlighted the risks of myopic decision-making, specifically the “Greedy Trap” in A²-RSR. This raises the question of whether a strategy can optimize solution quality without the instability of greedy heuristics or the computational cost of global rescheduling. This section demonstrates that the proposed ERRE strategy offers a viable solution. By performing coordinated optimization within a defined Affected Scope, ERRE provides sufficient flexibility to identify high-quality solutions while minimizing schedule disruption and computational overhead. This approach aims to balance solution quality, stability, and computational efficiency.

5.3.1. Robustness of ERRE: Effectively avoiding the “greedy trap”

To evaluate ERRE’s ability to mitigate the “Greedy Trap”, we compare its makespan performance against A²-RSR. Fig. 6 presents this comparison across the four representative instances.

The results indicate that ERRE consistently outperforms the greedy baseline. Across 28 test scenarios, ERRE surpasses A²-RSR in 27 cases. Its effectiveness is most evident on the MK10 instance, where A²-RSR showed significant degradation. For example, in the MK10-S3 scenario, while A²-RSR’s makespan increased to 417.14, ERRE maintained it at 274.52—a 34.2% improvement. This contrast demonstrates ERRE’s reliability in preventing decision failures.

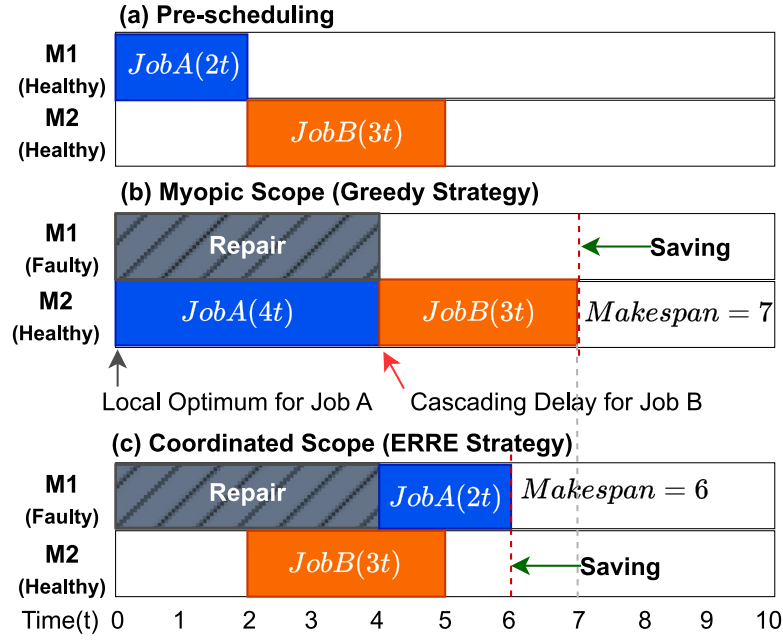


Fig. 5. Schematic of the “Greedy Trap”. Note that M1 is the faster resource for Job A. (b) **Myopic Strategy**: Job A greedily seizes the slower M2, blocking the critical path for Job B and extending the makespan to 7t. (c) **Coordinated Strategy**: Job A waits for the M1 repair, preserving M2 for Job B to achieve global optimality (6t).

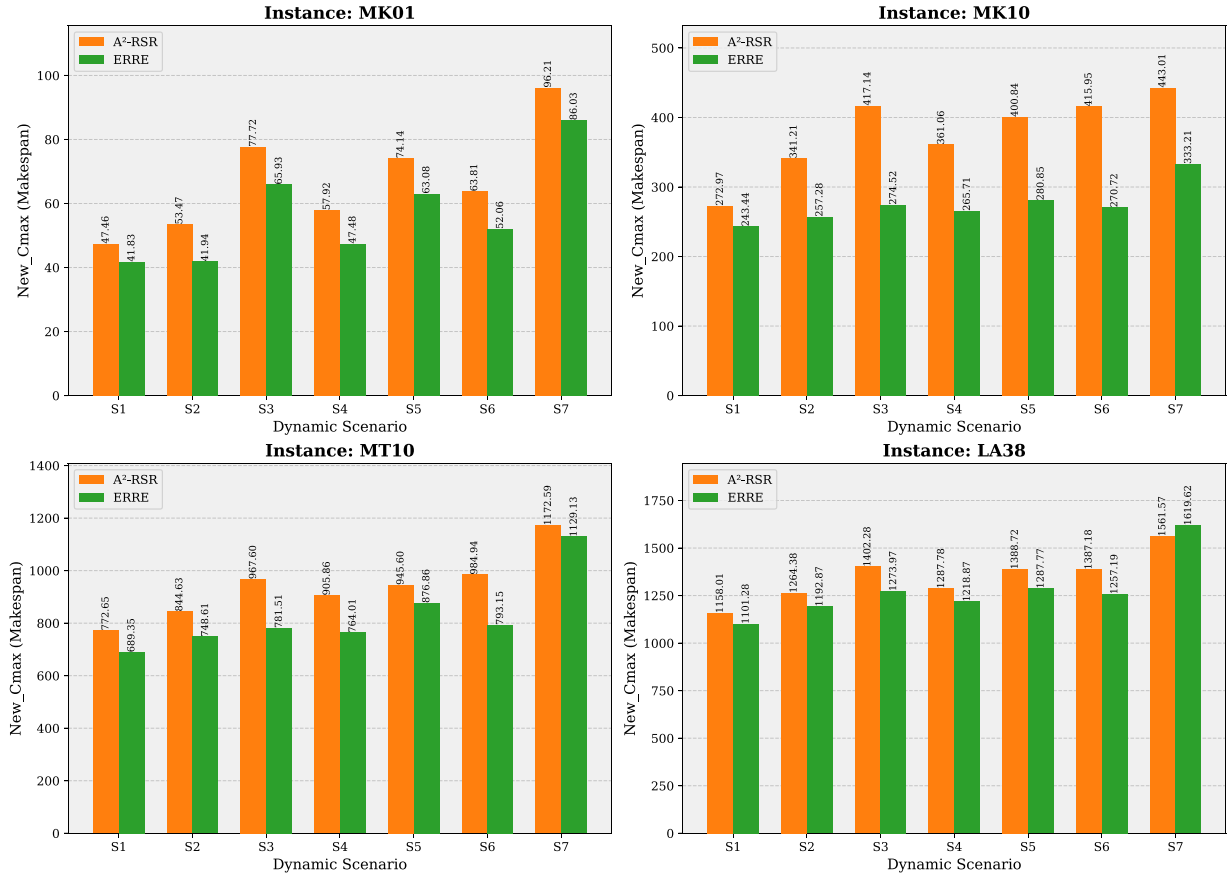


Fig. 6. Makespan comparison showcasing the superiority of ERRE over the greedy strategy A²-RSR.

The analysis also identifies the operational boundaries of ERRE. A²-RSR achieves a better makespan in only one case: the S7 (*Avalanche-Chaos*) scenario of the largest instance, LA38. In this scenario, the Affected Scope (Ω_A) becomes exceptionally large, creating a substantial

subproblem. Under strict time constraints, ERRE’s evolutionary search may not fully converge, resulting in a solution (1619.62) marginally inferior to the greedy result (1561.57). However, this is an isolated case. In other complex scenarios, such as MT10-S7, ERRE maintains

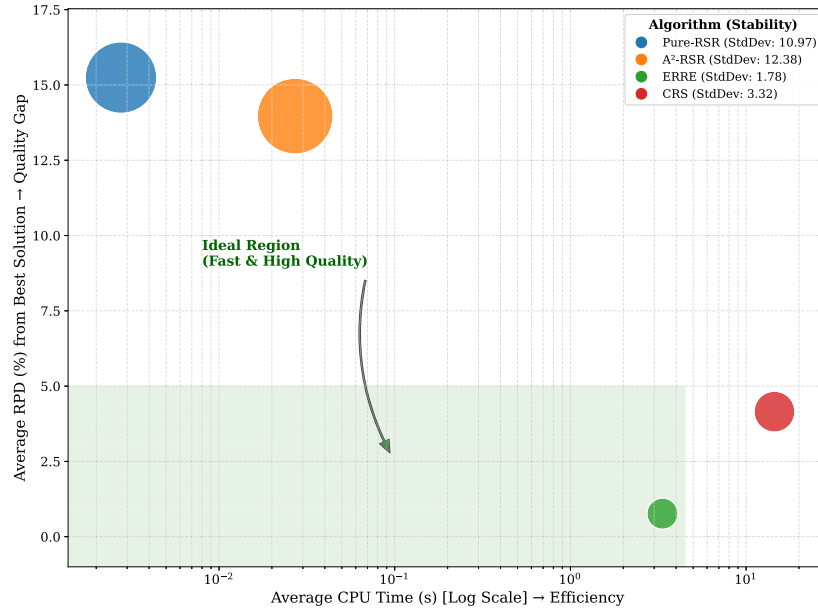


Fig. 7. Aggregate performance of four rescheduling strategies. The X-axis represents efficiency (CPU time), the Y-axis denotes quality (M-RPD), and the bubble size indicates the instability of solution quality (RSD-RPD).

a clear advantage (1129.13 vs 1172.59), confirming its robustness in the majority of dynamic environments.

The robustness of ERRE results from its coordinated optimization mechanism. Unlike the single-operation decisions of A²-RSR, ERRE optimizes over the entire set of affected operations. This broader decision horizon allows it to anticipate and mitigate the downstream consequences of resource allocation, such as potential bottlenecks. By considering interdependencies within the Affected Scope, ERRE avoids the conditions that lead to the “Greedy Trap”.

5.3.2. Comprehensive effectiveness: Balancing quality, efficiency, and stability

Beyond robustness, ERRE demonstrates superior overall effectiveness. To visualize the trade-offs involved, Fig. 7 presents an aggregated performance bubble chart that integrates efficiency (CPU Time) into the analysis, synthesizing results from all 70 test scenarios.

The positioning of the four strategies in Fig. 7 illustrates distinct performance profiles. Pure-RSR and A²-RSR cluster in the top-left area, exhibiting high speed but poor solution quality (high Y-axis position). Notably, they display the largest bubbles, visualizing extreme quality instability. This confirms that while greedy strategies are fast, their performance fluctuates wildly (high standard deviation), making them prone to severe degradation when falling into local optima like the “Greedy Trap”. Conversely, CRS is positioned on the far right, indicating significantly higher computational costs.

ERRE stands out as the only strategy within the “Ideal Region”. Its position highlights advantages across all three dimensions. Regarding efficiency, it is located to the left of CRS, maintaining on-line responsiveness (approx. 2–4 s). Regarding quality, ERRE is positioned vertically lower than CRS. This suggests that by focusing evolutionary search on the relevant subproblem (Ω_A), ERRE converges to higher-quality solutions more effectively than the global strategy under tight time constraints. Finally, ERRE exhibits the smallest bubble size among all strategies. Unlike the erratic greedy methods, ERRE delivers consistently high-quality results across all diverse test scenarios, demonstrating superior performance stability.

In summary, ERRE effectively balances the trade-offs. It avoids the instability of greedy rules and the inefficiency of global rescheduling, providing a solution that is simultaneously fast, stable, and high-quality.

5.4. Ablation study: Validating the optimization scope mechanism

To validate the contribution of the ASIA mechanism, we conducted an ablation study isolating the impact of the Optimization Scope (Ω) from the algorithmic logic. We compared the proposed ERRE (operating on the Affected Scope Ω_A) against the benchmark variant ERRE-Full (operating on the Global Future Scope Ω_F).

Experimental Design: As detailed in Section 4.3 and Table 5, both strategies share identical Memetic Algorithm kernel and parameter configurations. The sole difference lies in the input scope provided to the solver. The evaluation consists of two phases:

1. **General Validation:** Using standard benchmark instances (Table 7) to test pruning accuracy in typical scenarios.
2. **Structural Stress Test:** Using the specialized *Decoupled* 10×10 instance (introduced in Section 4.1) to visualize the “Scope-Complexity Trade-off” in an environment where disruption propagation is structurally limited.

5.4.1. Performance on standard benchmarks: Validation of effective pruning

Table 7 presents the comparative results on representative standard instances. The data confirms the Effective Pruning capability of the ASIA mechanism.

In the majority of scenarios (e.g., LA38-S4, LAR04-S3), ERRE achieves a makespan identical to ERRE-Full ($\Delta C_{max} = 0$). This empirical evidence indicates that the operations pruned by ASIA ($\Omega_F \setminus \Omega_A$) are practically irrelevant to the recovery of the global objective. By filtering out these non-critical operations, ERRE improves computational efficiency—reducing CPU time by approximately 10% in large-scale instances like LAR04—without compromising solution quality.

5.4.2. The “focus advantage” in decoupled scenarios

In specific complex cases (e.g., LA38-S1 in Table 7), the local strategy ERRE outperforms the global strategy ERRE-Full. To investigate this, we analyzed the results from the Decoupled Stress Test, as visualized in Fig. 8.

The results reveal two critical insights regarding the “Scope-Complexity Trade-off”:

1. **Superior Convergence (Better Quality):** ERRE achieved a significantly lower makespan (172.12) compared to ERRE-Full (186.87).

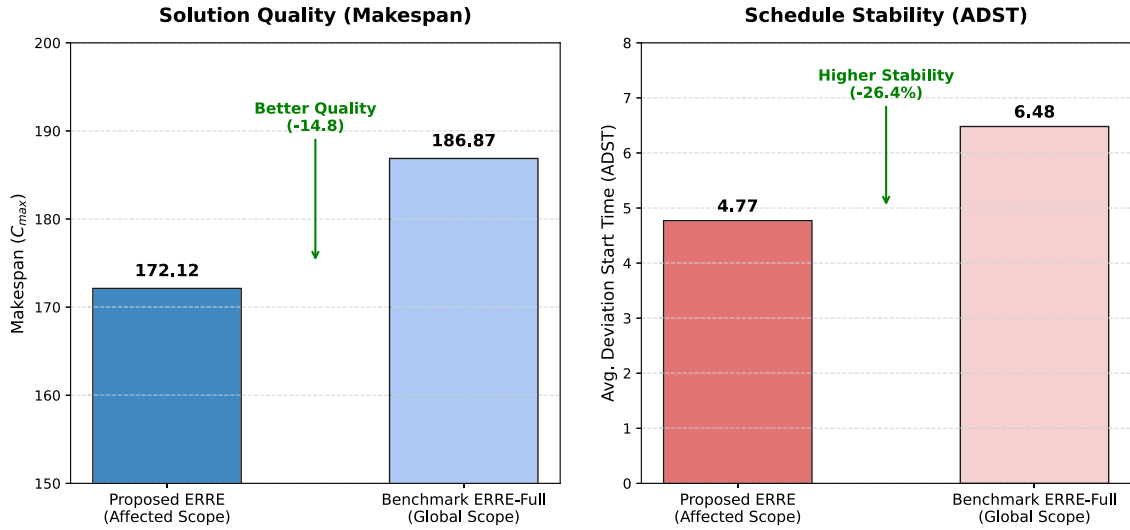


Fig. 8. Comparison on the Decoupled_10 × 10 Instance. A fault occurs in Machine Group A. ERRE significantly outperforms ERRE-Full in both solution quality (lower Makespan) and stability (lower ADST).

Table 7

Ablation results on standard benchmarks. Negative Δ indicates ERRE performs better.

Inst.	Sce.	ERRE (Ω_A)		ERRE-Full (Ω_F)		Difference (Δ)		
		C_{max}	Time (s)	C_{max}	Time (s)	ΔC_{max}	$\Delta Time$	Result
LA38	S1	1101.28	1.34	1110.08	1.53	-8.80	-0.19	Better
LA38	S4	1218.87	4.60	1218.87	5.00	0.00	-0.40	Same
MT06	S1	49.94	0.20	50.12	0.23	-0.18	-0.03	Better
MT06	S7	105.29	0.82	105.29	0.87	0.00	-0.05	Same
LAR04	S7	492.95	19.67	492.95	21.63	0.00	-1.96	Faster

Since both algorithms operate with the same fixed computational budget, ERRE-Full is constrained by the “curse of dimensionality”. Its genetic search is distributed across the entire global scope, allocating evaluations to unaffected operations (Group B). In contrast, ERRE concentrates evolutionary pressure solely on the affected Group A, enabling faster convergence to a near-optimal solution.

2. Enhanced Stability (Lower ADST): The ADST metric highlights the implicit cost of the global scope. ERRE-Full exhibits a higher deviation (6.48 vs. 4.77). This excess deviation arises because the stochastic nature of the GA reschedules operations in the unaffected Group B, causing “schedule nervousness”. ERRE, constrained by ASIA, isolates Group B from changes, resulting in a robust schedule with minimal disruption.

The ablation study confirms that ASIA improves performance beyond heuristic acceleration. By strategically defining the Optimization Scope, it simultaneously enhances efficiency, solution quality, and system stability, validating the structural advantage of ERRE over traditional global rescheduling approaches.

In summary, the ablation study confirms that the proposed scope definition mechanism does not merely accelerate computation. By isolating unaffected operations from the stochastic nature of metaheuristic search, ERRE simultaneously enhances *temporal stability* and *quality stability*, validating its structural advantage over unconstrained global search.

5.5. The boundaries of global optimization: CRS as a reference baseline

The design of CRS serves as a reference baseline representing the traditional global rescheduling paradigm (Ω_F). In this study, to ensure a rigorous comparison within industrial real-time constraints, the computational budget for CRS was standardized at $N_p = 50$ and $Gen = 50$.

As detailed in Table 6, while CRS identifies competitive solutions in small-scale or tightly coupled instances like MK10 (prevailing in scenarios S1, S2, and S4), its overall performance is significantly eclipsed by the proposed ERRE strategy as problem complexity escalates.

5.5.1. Efficiency gap in high-dimensional search

Fig. 9 provides a direct comparison of makespan performance, revealing an “eroding advantage” of global optimization as the scheduling horizon and machine flexibility increase. At the small-scale level (MT06, MK10), CRS maintains a marginal edge in specific cases—such as MK10-S1 (243.01 vs. 243.44)—where the global search space remains relatively manageable even with a restricted population size. Conversely, as the problem scale shifts to medium and large instances (MT10, LAR04), ERRE demonstrates absolute dominance, securing the best solution in 100% of these 14 scenarios. For instance, in MT10-S1, ERRE (689.35) outperforms CRS (728.62) by a significant margin, illustrating a clear performance inversion as a function of problem dimensionality.

This disparity stems from the inherent efficiency of resource allocation. In a global scope (Ω_F), the search space undergoes a combinatorial explosion with the addition of each job. As the sensitivity analysis in Appendix B demonstrates, while CRS *could* theoretically improve its solution quality by expanding N_p to 500, the resulting 40-fold increase in CPU time (exceeding 13 min for LAR03) is prohibitive for online rescheduling. Consequently, when strictly bounded by real-time latency requirements, the global search of CRS becomes “too broad and shallow”. In contrast, ERRE concentrates its limited computational budget on the precision-targeted affected scope (Ω_A), facilitating a deeper and more converged search that consistently outperforms an unconverged global exploration.

5.5.2. The real-time bottleneck of global rescheduling

The most critical bottleneck of CRS is its lack of scalability. Fig. 10 plots the CPU times on a logarithmic scale, revealing a stark divergence in computational trajectories. ERRE responds almost instantaneously for small instances and caps at approximately 20 s even for the most intensive LAR04-S7 scenario. In contrast, the runtime of CRS scales poorly, escalating rapidly towards 90 s.

As justified in Section 4.3 and Appendix B, although global optimization is theoretically robust, it is practically marginalized by the latency-quality trade-off. For the LAR04 instance, the time investment of CRS is 4.5 times that of ERRE, yet it yields inferior results. This confirms that for dynamic FJSP, global optimization is not merely

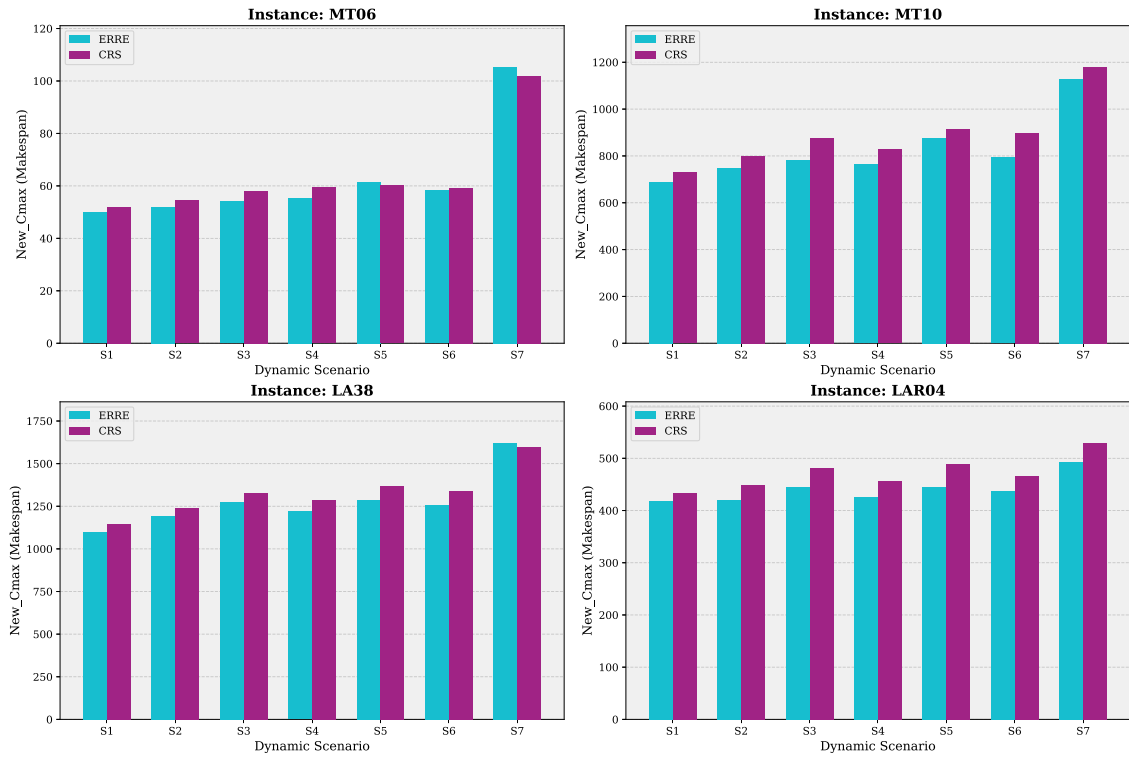


Fig. 9. Makespan comparison between ERRE and CRS, highlighting the diminishing effectiveness of global search under restricted computational budgets.

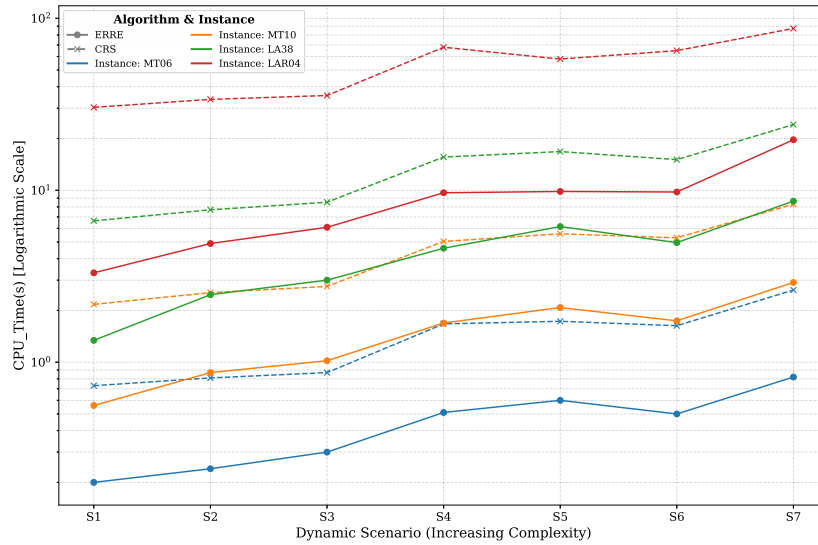


Fig. 10. Comparison of computational time between ERRE and CRS on a logarithmic scale, illustrating the scalability advantage of the local scope.

slower but functionally inefficient under time-critical conditions. By the time a global search identifies a superior solution (if at all), the physical shop-floor state may have already transitioned, rendering the calculated schedule obsolete.

In summary, this analysis delineates the practical boundaries of global optimization. CRS functions as an Approximate Quality Reference (AQR) that underscores the high overhead of maintaining a global perspective. The strategic value of ERRE lies in its ability to “right-size” the optimization problem, achieving state-of-the-art solution quality

with a computational footprint that remains strictly within the window of industrial feasibility.

5.6. Discussion: Trade-offs in optimization scope

The analyses of solution quality (makespan), efficiency (CPU Time), and stability (ADST) illustrate a core principle of dynamic scheduling: no single strategy dominates across all performance dimensions. The challenge lies in reconciling the trade-offs among these conflicting

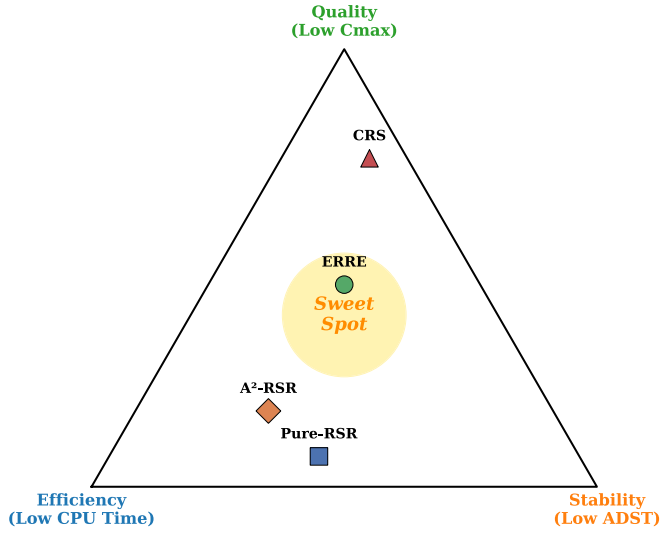


Fig. 11. Positioning of four rescheduling strategies in the quality-efficiency-stability trade-off space.

objectives. This study identifies that the Optimization Scope is the critical mechanism for regulating this balance.

To synthesize these findings, we propose the “Quality-Efficiency-Stability Triangle” shown in Fig. 11. The three vertices represent the idealized extremes: perfect solution quality, instantaneous computation, and absolute schedule stability. A strategy’s position reflects its inherent design trade-off. The central “sweet spot” represents the optimal equilibrium among all three objectives.

Based on the empirical evidence from Sections 5.1–5.4, we position the four strategies within this trade-off space:

- **Pure-RSR** (Ω_0): Anchored near the efficiency vertex. As a passive repair method, it offers maximum *sequence stability* (no machine changes) but suffers from poor *temporal stability* due to delay propagation, thereby sacrificing opportunities for quality improvement.
- **A²-RSR** (Ω_G): Biased towards efficiency. It attempts to improve quality at minimal cost, but as the “Greedy Trap” analysis revealed, this myopic approach leads to instability, representing a high-risk strategy.
- **CRS** (Ω_F): Targeting the quality vertex. Theoretically, it aims for the global optimum. However, this positioning represents its *potential* rather than its *realized performance* under constraints. Under strict time limits, it often fails to converge fully, rendering it computationally prohibitive and inefficient, sacrificing *sequence stability* without guaranteeing superior quality in practice.

ERRE (Ω_A) occupies the central “sweet spot”. This positioning represents the core contribution of this work. It outperforms myopic strategies (Pure-RSR and A²-RSR) by delivering superior quality and robustness with a controlled computational investment. Furthermore, it proves more effective than CRS, achieving a better average solution quality (M-RPD of 0.76% vs. 4.15%) while maintaining significantly higher efficiency. As validated by the ablation study (Section 5.4), this efficiency is not merely a heuristic acceleration but the result of the ASIA mechanism’s pruning capability—filtering out non-critical operations that otherwise disperse search effort. This confirms that effectively navigating the trade-off space (the “sweet spot”) yields better practical outcomes than exclusively targeting the theoretical quality optimum (the CRS vertex). Regarding stability, while CRS demonstrates strong temporal stability by compressing delays through global re-optimization, its high computational cost renders it impractical for

real-time response. ERRE, by balancing these factors, achieves a stability level competitive with global optimization but with a fraction of the computing time (e.g., 19.7 s vs. 87.3 s in extreme cases), avoiding the significant temporal deviations observed in passive repair strategies.

Ultimately, this investigation reveals a guiding principle for selecting an Optimization Scope: a scope that is too small (e.g., Ω_G) risks making detrimental decisions due to a lack of perspective; a scope that is too large (e.g., Ω_F) is constrained by computational complexity. The ERRE strategy, through its identification of the Affected Scope (Ω_A), addresses this dilemma. By “right-sizing” the problem, it performs focused optimization on the critical subproblem. This approach allows it to strike a robust balance among quality, efficiency, and stability, establishing its value as a practical rescheduling strategy.

Generalizability to other disruption types. While the experimental analysis in this study focused on machine breakdowns, the proposed optimization-scope framework offers a generalizable framework applicable to other common manufacturing disruptions. The hierarchical classification ($\Omega_0 \rightarrow \Omega_F$) remains structurally valid regardless of the specific trigger event; only the identification logic for the Affected Scope (Ω_A) requires adaptation. For instance, in the case of Rush Order Arrivals, Ω_A would naturally encompass the new job, the conflicted operations on the target machines, and any downstream tasks affected by the schedule shift. Similarly, for Material Delays, the scope would propagate downstream along the precedence constraints of the delayed job. In all these scenarios, the fundamental trade-off identified in Fig. 11 holds: myopic adjustments (Ω_G) risk blocking critical paths (triggering the “Greedy Trap”), while global rescheduling (Ω_F) incurs prohibitive computational costs. Therefore, the core principle of “right-sizing” the optimization boundary via Ω_A serves as a foundational design principle for dynamic scheduling systems.

6. Conclusion and future work

This paper investigated the impact of optimization scope on dynamic rescheduling. The findings indicate that scope definition is a critical factor in balancing solution quality, efficiency, and stability. While myopic greedy strategies (Ω_G) are prone to “Greedy Traps” and global optimization (Ω_F) is constrained by combinatorial complexity, the proposed Event-driven Rescheduling with Elite Evolution (ERRE) strategy effectively reconciles these extremes by targeting a specific Affected Scope (Ω_A). The ablation study confirmed that this targeted scope definition is the primary mechanism driving performance, enabling superior convergence by isolating critical decision variables. Empirical results from 70 dynamic scenarios demonstrate that ERRE outperforms the global rescheduling strategy (CRS) in 52 cases, achieving a substantially lower Mean Relative Percentage Deviation (M-RPD) of 0.76% compared to 4.15% for CRS. Furthermore, ERRE exhibits superior scalability: while the computation time for global optimization reached nearly 90 s for large-scale instances, ERRE maintained an average response time of 3.5 s, with a maximum of approximately 20 s. These results substantiate that a fully converged search within a relevant sub-problem is superior to a shallow search across the global space under strict time constraints.

From a managerial perspective, particularly for deployment in Manufacturing Execution Systems (MES), the proposed strategy offers distinct advantages regarding data dependency, latency predictability, and system reliability. Specifically, focusing on the Affected Scope alleviates the data synchronization burden. In contrast to global rescheduling, which necessitates a real-time snapshot of the entire shop floor, ERRE requires state data only from the disrupted machine group and related job sequences. Regarding latency, the bounded nature of the Affected Scope ensures predictable execution times, preventing computation delays that could violate real-time control cycles. However, we acknowledge a practical limitation in extremely saturated environments where tight coupling causes the Affected Scope (Ω_A) to expand toward

the Global Scope (Ω_F). While solution quality remains robust, the computational efficiency advantage over global rescheduling may diminish in these high-density scenarios. Finally, to guarantee system reliability, we recommend a two-tier strategy: utilizing ERRE as the primary solver for quality optimization, while maintaining deterministic Right-Shift Rescheduling (RSR) as a fallback mechanism to ensure feasibility in the event of solver timeouts.

Future research will focus on transitioning from static scope definitions to dynamic adaptation. We propose three promising directions:

- **Adaptive Scope Control:** Developing a mechanism to adjust the Optimization Scope size in real-time based on disruption severity. This could involve rule-based logic or reinforcement learning to expand the scope for severe disruptions and contract it for minor perturbations.
- **Co-evolution of Scope and Schedule:** Treating the Optimization Scope as a decision variable within an evolutionary algorithm. This approach would allow the solver to simultaneously determine the optimal schedule and the optimal scope configuration for generating that schedule.
- **Graph Learning for Scope Prediction:** Employing Graph Neural Networks (GNNs) to predict the effective Optimization Scope from the system state. By learning topological dependencies between jobs and machines, a model could recommend a scope *a priori*, reducing the reliance on iterative search procedures.

CRedit authorship contribution statement

Ruirui Sun: Writing – original draft, Visualization, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Guanghe Cheng:** Methodology, Funding acquisition, Conceptualization. **Qingyan Ding:** Supervision, Resources, Conceptualization. **Xiaojie Zhao:** Validation, Data curation.

Acknowledgments

The authors would like to thank Dr. Fengqi Hao for his assistance in validating the algorithm's implementation and Dr. Huijuan Hao for her advice on parameter standardization and language refinement during the revision process. We also express our sincere gratitude to the Editor and the anonymous reviewers for their constructive comments and valuable suggestions, which have significantly improved the quality of this manuscript. This work was supported by the Shandong Provincial Technology Innovation System Project, China (Grant No. YDZX2024121).

Appendix A. Algorithm validation on benchmark instances

To assess the performance of the Genetic Algorithm used in the CRS strategy, we evaluated it on the standard Brandimarte benchmark instances (Mk01-Mk10). Table A.8 presents the comparison between the Best Known Solution (BKS) and the results obtained by the proposed algorithm.

Appendix B. Parameter sensitivity analysis for population size

Genetic Algorithm performance depends on parameter settings, particularly population size (N_p), which influences both solution quality and convergence speed. To justify the selection of parameters ($N_p = 50$) for dynamic rescheduling and address concerns regarding premature convergence, this section presents a sensitivity analysis.

We selected two representative instances, a medium-scale (MK10) and a large-scale (LAR03), testing them under the complex concurrent failure scenario (S5). We compared three configurations: the setting

Table A.8

Experimental results on Brandimarte instances.

Instance	Size (Jobs $\times$ Mach)	BKS	Best found	Gap (%)	Avg. time (s)
Mk01	10 $\times$ 6	40	40	0.00	61.28
Mk02	10 $\times$ 6	26	28	7.69	60.83
Mk03	15 $\times$ 8	204	204	0.00	196.61
Mk04	15 $\times$ 8	60	65	8.33	104.48
Mk05	15 $\times$ 4	172	173	0.58	125.61
Mk06	10 $\times$ 15	57	64	12.28	209.19
Mk07	20 $\times$ 5	139	144	3.60	118.61
Mk08	20 $\times$ 10	523	523	0.00	326.23
Mk09	20 $\times$ 10	307	307	0.00	344.76
Mk10	20 $\times$ 15	197	218	10.66	341.87

Table B.9

Trade-off analysis between solution quality and computational cost under different population sizes (Scenario S5, Avg. of 10 runs).

Instance	Cfg.	Params		Quality (C_{max})		Cost (Time)	
		N_p	Max.Gen	Value	Imp. (%)	Time (s)	Ratio ($\times$)
MK10	A (Ours)	50	50	283.6	–	17.4	1.0
	B	200	100	269.7	4.9	138.7	8.0
	C	500	200	257.6	9.2	692.6	39.9
LAR03	A (Ours)	50	50	284.0	–	20.5	1.0
	B	200	100	270.6	4.7	163.8	8.0
	C	500	200	264.9	6.7	791.1	38.5

used in this paper (Config A), a medium setting (Config B), and a high-computation setting common in static scheduling literature (Config C, see Fan et al., 2024; Li & Gao, 2016). Each experiment was run independently 10 times to mitigate stochastic variations.

The results in Table B.9 indicate diminishing returns:

1. **Limited Quality Improvement:** Increasing the population size from 50 to 500 (Config C) yielded quality improvements of only 9.2% (MK10) and 6.7% (LAR03). This suggests that $N_p = 50$ captures a substantial portion of the optimization potential.
2. **Computational Cost Increase:** To achieve this < 10% improvement, the computational time increased by nearly 40 times. For LAR03, the rescheduling time increased from 20.5 s to over 13 min (791 s).

In dynamic manufacturing environments, a 13 minute delay is impractical; the incurred Downtime Cost would likely exceed the benefits of a 6.7% schedule optimization. In contrast, Config A maintains the response time within 21 s, satisfying requirements for Near Real-time Response. Thus, $N_p = 50$ represents an effective trade-off between engineering practicality and solution quality.

Appendix C. Extended computational results on additional instances

Table C.10 presents the detailed performance metrics for the benchmark instances that were not displayed in Section 5.1 (Table 6). To avoid redundancy, data points already reported in the main text are omitted here. These extended results further validate the robustness of the proposed strategy across a wider range of problem scales and complexities.

Data availability

No data was used for the research described in the article.

Table C.10

Performance comparison on additional benchmark instances. Bold values indicate the best result..

Ins.	Sce.	Metrics (New Cmax/ADST/CPU Time (s))			
		Pure-RSR	A ² -RSR	ERRE	CRS
Kacem1	S1	17.26/1.61/< 0.01	15.46/1.19/< 0.01	14.64/0.35/0.10	14.11/0.70/0.33
	S2	31.11/3.10/< 0.01	19.78/2.53/< 0.01	21.78/0.97/0.09	21.87/2.36/0.35
	S3	42.70/13.33/< 0.01	39.24/1.86/< 0.01	35.41/0.98/0.12	37.23/5.01/0.37
	S4	31.12/6.12/< 0.01	23.85/2.33/< 0.01	23.15/0.67/0.18	24.77/3.24/0.71
	S5	47.54/17.84/< 0.01	36.17/3.57/< 0.01	34.15/2.07/0.21	36.64/7.93/0.76
	S6	29.14/8.64/< 0.01	24.79/3.07/< 0.01	23.48/1.39/0.20	24.94/4.29/0.73
	S7	80.76/33.03/< 0.01	76.79/6.96/< 0.01	74.67/3.38/0.32	81.15/30.73/1.13
Kacem2	S1	17.71/0.96/< 0.01	19.66/0.90/< 0.01	14.47/0.08/0.16	18.02/1.22/0.62
	S2	31.27/2.74/< 0.01	24.98/1.67/< 0.01	25.73/0.45/0.20	27.20/2.51/0.69
	S3	52.51/10.01/< 0.01	44.60/2.63/< 0.01	44.39/2.74/0.23	45.37/7.89/0.73
	S4	35.23/6.14/< 0.01	28.38/2.28/< 0.01	27.25/2.07/0.40	27.87/4.27/1.38
	S5	55.82/14.62/< 0.01	45.40/3.37/< 0.01	47.08/3.41/0.49	43.59/8.77/1.46
	S6	33.07/6.56/< 0.01	27.06/3.45/< 0.01	28.23/3.00/0.46	28.24/5.24/1.42
	S7	104.48/40.59/< 0.01	93.67/5.21/< 0.01	90.64/9.87/0.70	90.68/23.73/2.19
Kacem3	S1	13.48/0.61/< 0.01	11.32/0.45/< 0.01	12.11/0.02/0.16	11.30/0.59/0.63
	S2	24.09/2.26/< 0.01	19.75/0.73/< 0.01	21.72/0.13/0.24	21.74/1.14/0.70
	S3	45.46/4.76/< 0.01	37.24/0.88/< 0.01	37.78/0.52/0.26	39.07/4.19/0.74
	S4	28.51/3.50/< 0.01	23.00/0.85/< 0.01	22.03/0.26/0.49	22.59/1.82/1.38
	S5	45.50/8.07/< 0.01	34.43/1.46/< 0.01	40.27/1.03/0.48	36.71/5.68/1.48
	S6	24.88/3.02/< 0.01	22.16/1.16/< 0.01	24.24/0.65/0.43	22.39/2.15/1.40
	S7	84.27/15.81/< 0.01	77.59/1.43/< 0.01	76.86/1.29/0.71	82.03/12.42/2.19
Kacem4	S1	19.22/0.38/< 0.01	15.52/0.30/< 0.01	16.07/0.01/0.27	15.96/0.44/1.20
	S2	31.45/2.39/< 0.01	25.26/1.05/< 0.01	23.23/0.62/0.38	23.63/1.69/1.28
	S3	52.00/14.96/< 0.01	43.41/2.47/< 0.01	40.30/2.33/0.53	43.34/6.92/1.40
	S4	34.22/4.30/< 0.01	27.20/1.19/< 0.01	27.77/1.35/0.79	26.09/2.30/2.50
	S5	51.07/13.04/< 0.01	41.01/2.90/< 0.01	43.25/3.40/1.00	40.28/7.50/2.77
	S6	32.55/6.80/< 0.01	26.45/2.68/< 0.01	28.04/2.32/0.89	26.42/3.91/2.65
	S7	93.77/33.29/< 0.01	84.41/3.79/< 0.01	86.11/10.62/1.52	85.42/18.66/4.22
MK01	S1	45.09/0.40/< 0.01	47.46/0.51/< 0.01	41.83/0.08/0.27	43.86/0.35/1.16
	S2	51.30/2.82/< 0.01	53.47/2.04/< 0.01	41.94/0.60/0.31	44.19/2.55/1.22
	S3	67.86/10.35/< 0.01	77.72/12.91/< 0.01	65.93/10.34/0.45	60.73/8.43/1.36
	S4	52.38/3.30/< 0.01	57.92/3.71/< 0.01	47.48/1.43/0.65	49.87/3.98/2.47
	S5	68.59/14.38/< 0.01	74.14/11.38/< 0.01	63.08/8.41/0.82	58.98/9.29/2.72
	S6	57.15/6.52/< 0.01	63.81/7.48/< 0.01	52.06/4.13/0.72	52.95/6.81/2.57
	S7	98.30/27.15/< 0.01	96.21/19.72/< 0.01	86.03/15.93/1.18	92.03/23.53/4.04
MK02	S1	31.29/0.42/< 0.01	33.10/0.66/< 0.01	29.26/0.16/0.30	30.85/0.58/1.12
	S2	40.95/4.40/< 0.01	45.47/3.25/< 0.01	31.70/1.63/0.43	34.53/2.67/1.28
	S3	53.55/14.94/< 0.01	60.79/9.32/< 0.01	44.26/6.22/0.53	46.18/8.47/1.44
	S4	42.24/5.31/< 0.01	50.36/4.46/< 0.01	37.25/2.96/0.87	39.38/4.12/2.58
	S5	52.46/15.36/< 0.01	63.31/10.20/< 0.01	45.13/7.61/1.03	49.22/10.06/2.84
	S6	44.76/8.77/< 0.01	64.19/9.40/< 0.01	39.54/3.95/0.86	41.10/5.64/2.63
	S7	83.64/28.88/< 0.01	83.80/18.70/< 0.01	76.10/20.16/1.48	77.91/22.44/4.22
MK03	S1	215.21/1.72/< 0.01	230.44/2.57/0.01	204.29/1.03/0.82	208.14/1.77/3.66
	S2	233.51/10.29/< 0.01	277.51/15.80/0.01	204.70/8.27/1.16	212.52/8.73/4.27
	S3	274.39/48.67/< 0.01	340.25/50.28/0.01	235.60/25.77/1.64	229.40/24.47/4.75
	S4	245.01/16.76/< 0.01	288.91/18.30/0.02	213.43/9.25/2.37	222.32/11.37/8.57
	S5	271.11/42.32/< 0.01	335.42/47.06/0.02	232.36/26.18/3.11	243.86/29.50/9.62
	S6	256.81/26.09/< 0.01	341.21/49.77/0.02	226.70/19.03/2.48	235.38/23.59/8.69
	S7	358.67/100.13/< 0.01	375.20/63.03/0.01	296.66/43.29/4.23	284.91/59.39/14.40
MK04	S1	69.33/0.83/< 0.01	72.77/1.22/< 0.01	65.56/0.21/0.49	68.44/0.94/1.98
	S2	74.41/4.43/< 0.01	90.40/7.00/< 0.01	67.36/2.91/0.63	70.69/4.96/2.31
	S3	95.59/16.41/< 0.01	102.18/15.03/< 0.01	85.70/10.18/0.76	80.05/11.24/2.52
	S4	84.39/7.01/< 0.01	93.99/8.41/< 0.01	75.36/4.80/1.27	77.38/7.10/4.61
	S5	98.62/18.35/< 0.01	100.17/14.54/< 0.01	86.40/11.28/1.45	84.68/12.63/4.93
	S6	91.76/13.05/< 0.01	103.40/16.80/< 0.01	78.30/7.74/1.31	78.93/10.84/4.66
	S7	137.24/41.84/< 0.01	127.31/24.74/< 0.01	122.69/21.22/2.16	125.05/30.68/7.44
MK05	S1	182.37/1.06/< 0.01	187.56/2.09/< 0.01	179.03/1.17/0.63	180.96/1.78/2.37
	S2	194.68/6.18/< 0.01	200.99/12.82/0.02	185.96/6.80/0.81	187.77/9.35/2.77
	S3	222.49/36.68/< 0.01	226.32/34.14/< 0.01	196.67/19.54/1.09	204.04/24.96/3.15
	S4	199.24/11.30/< 0.01	209.37/15.85/< 0.01	194.32/10.32/1.75	195.32/11.43/5.60
	S5	221.64/38.70/0.01	237.28/37.14/0.02	204.74/25.26/2.21	209.90/28.85/6.38
	S6	214.48/19.51/< 0.01	238.32/37.34/0.01	196.04/17.53/1.86	199.31/23.09/5.70
	S7	279.63/76.99/< 0.01	291.91/64.64/0.01	256.63/60.04/3.35	260.10/59.48/9.50

(continued on next page)

Table C.10 (continued).

Ins.	Sce.	Metrics (New Cmax/ADST/CPU Time (s))			
		Pure-RSR	A ² -RSR	ERRE	CRS
MK06	S1	75.82/1.78/< 0.01	73.73/1.09/< 0.01	71.59/0.97/1.05	69.92/0.63/4.03
	S2	83.12/5.34/< 0.01	79.42/2.48/< 0.01	74.56/3.01/1.37	75.02/3.70/4.73
	S3	100.87/24.92/< 0.01	96.52/12.00/< 0.01	84.69/9.46/1.76	88.98/12.25/5.08
	S4	84.82/6.16/< 0.01	80.88/2.72/< 0.01	81.58/5.06/2.82	78.50/3.35/8.80
	S5	96.09/19.73/< 0.01	104.31/13.18/0.01	88.73/11.08/2.86	86.75/11.74/7.65
	S6	80.26/9.75/< 0.01	88.96/10.36/< 0.01	79.62/7.26/1.74	81.82/7.89/5.08
	S7	139.74/47.94/< 0.01	122.15/22.12/0.01	117.52/23.87/4.24	118.24/26.42/9.48
MK07	S1	158.82/1.78/< 0.01	164.75/1.73/< 0.01	154.16/1.43/0.59	155.79/1.64/2.31
	S2	177.64/9.66/< 0.01	200.98/16.98/< 0.01	159.25/8.72/0.82	163.60/10.88/2.70
	S3	211.70/31.92/< 0.01	227.39/33.27/0.01	180.70/22.32/1.00	200.62/29.22/2.98
	S4	188.51/16.09/< 0.01	203.04/17.39/0.01	171.35/11.18/1.61	177.16/10.71/5.20
	S5	213.03/45.12/< 0.01	229.17/37.96/0.01	191.42/27.16/1.96	192.98/31.15/5.94
	S6	215.11/26.43/< 0.01	223.98/36.34/0.02	180.49/17.30/1.60	179.05/22.15/5.31
	S7	295.35/81.65/< 0.01	304.97/62.44/< 0.01	261.99/61.44/2.95	278.95/56.91/8.55
MK08	S1	527.07/1.01/< 0.01	553.56/4.74/< 0.01	523.00/2.49/1.27	526.68/3.96/6.84
	S2	535.61/6.30/< 0.01	568.58/22.55/< 0.01	525.80/18.51/1.78	535.03/17.15/7.59
	S3	605.79/56.59/< 0.01	678.16/105.06/0.02	571.96/44.26/2.36	566.21/57.01/8.90
	S4	557.90/9.10/< 0.01	592.17/25.47/0.02	546.78/24.04/3.68	559.04/23.03/15.57
	S5	599.94/51.06/< 0.01	711.68/99.52/0.02	552.99/50.09/4.07	556.49/46.77/12.94
	S6	570.64/27.92/< 0.01	685.78/85.67/0.01	549.14/39.52/3.77	554.85/43.69/15.60
	S7	696.47/117.83/< 0.01	738.49/115.99/0.02	632.87/78.11/6.78	642.01/86.73/23.68
MK09	S1	319.93/1.51/< 0.01	338.29/2.80/0.01	315.18/2.02/1.43	318.39/2.84/7.68
	S2	336.89/7.56/< 0.01	397.20/24.68/0.01	328.90/13.25/2.04	334.59/17.00/8.80
	S3	379.85/50.81/< 0.01	442.95/61.79/0.02	368.77/39.32/2.72	375.70/40.80/9.55
	S4	351.14/14.88/< 0.01	393.14/23.02/0.03	340.90/19.32/4.46	344.83/18.33/17.84
	S5	386.57/53.89/< 0.01	463.88/77.62/0.03	364.76/40.49/5.48	375.63/45.24/19.26
	S6	365.60/25.74/< 0.01	449.65/66.09/0.01	351.42/29.70/4.40	352.28/32.05/17.46
	S7	497.73/136.71/< 0.01	516.11/86.92/0.04	447.42/83.83/8.41	427.32/69.51/28.40
MK11	S1	638.33/1.81/< 0.01	657.02/12.78/0.01	636.01/4.16/1.13	634.70/6.85/5.30
	S2	673.96/13.70/< 0.01	688.81/35.13/0.01	653.22/23.73/1.43	653.83/31.69/5.90
	S3	734.53/63.30/< 0.01	746.78/105.09/0.01	663.21/58.48/1.98	672.09/73.88/6.58
	S4	694.65/26.68/0.01	718.67/40.41/0.02	664.24/33.43/3.09	670.83/38.51/11.97
	S5	741.07/72.29/< 0.01	771.76/138.79/0.02	680.30/70.59/3.90	699.04/83.09/13.34
	S6	731.33/41.12/0.01	781.11/135.56/0.02	666.82/53.33/3.25	690.36/83.25/12.20
	S7	884.58/176.85/< 0.01	876.77/146.55/0.02	813.56/139.31/6.03	818.47/139.79/20.28
MK12	S1	522.71/1.59/< 0.01	551.16/5.01/0.01	509.10/3.87/1.07	521.81/5.02/5.66
	S2	526.81/7.94/< 0.01	584.42/24.73/0.01	522.99/18.33/1.33	524.04/29.78/6.43
	S3	632.98/74.07/< 0.01	664.57/84.05/0.01	580.66/54.54/1.79	588.09/69.05/6.98
	S4	557.88/17.82/0.01	618.69/41.75/0.02	535.31/28.63/2.87	554.69/24.82/12.53
	S5	632.87/73.52/0.01	675.40/95.71/0.03	600.19/64.24/3.49	578.82/71.53/14.06
	S6	572.72/32.41/< 0.01	664.36/87.59/0.02	559.83/50.66/3.03	562.00/58.17/12.93
	S7	731.40/135.87/0.02	750.69/109.05/0.03	701.00/107.97/5.20	679.40/108.73/21.01
MK13	S1	463.60/2.26/0.01	485.63/7.38/0.02	449.57/1.97/1.29	450.23/3.30/7.23
	S2	496.04/17.00/< 0.01	563.86/37.48/0.01	474.80/26.10/2.05	476.62/25.29/8.35
	S3	580.17/78.23/< 0.01	620.27/83.03/0.02	498.58/42.66/2.44	505.52/56.03/9.05
	S4	512.35/18.50/0.02	554.63/25.18/0.02	484.10/24.28/3.94	488.59/25.44/16.28
	S5	580.39/87.26/< 0.01	647.55/94.99/0.03	509.96/51.91/5.03	515.13/53.26/17.91
	S6	550.44/42.33/< 0.01	613.72/90.36/0.02	502.22/42.32/4.15	511.61/57.09/16.64
	S7	744.37/202.69/< 0.01	708.45/100.03/0.03	622.49/81.05/7.15	621.48/97.56/26.86
MK14	S1	702.44/0.87/< 0.01	743.43/6.13/0.01	695.70/3.82/1.51	702.80/3.32/9.52
	S2	729.87/9.60/< 0.01	801.18/38.62/0.01	694.00/19.17/1.92	703.55/21.37/10.52
	S3	819.88/82.61/< 0.01	849.10/110.42/0.02	749.47/67.53/2.70	757.50/70.71/11.70
	S4	754.82/14.36/< 0.01	831.23/53.25/0.03	704.24/14.09/3.63	724.31/26.71/21.14
	S5	826.05/87.09/< 0.01	844.05/107.62/0.03	745.34/73.25/5.33	735.06/70.77/23.20
	S6	772.02/40.74/< 0.01	851.38/107.04/0.03	743.65/54.59/4.06	728.99/73.61/21.21
	S7	956.56/180.33/< 0.01	899.29/119.26/0.04	845.62/118.24/8.00	813.07/112.44/35.22
MK15	S1	396.64/4.58/< 0.01	428.39/6.81/0.01	390.51/1.64/1.46	388.07/3.39/10.28
	S2	427.28/14.35/< 0.01	530.96/31.69/0.01	409.31/14.77/2.33	426.87/19.51/11.05
	S3	493.43/68.53/< 0.01	584.94/68.42/0.02	442.94/39.35/2.87	454.73/48.92/12.02
	S4	445.37/27.79/< 0.01	512.64/26.51/0.03	421.47/21.48/4.74	432.06/24.54/22.37
	S5	492.33/75.36/< 0.01	597.01/68.20/0.03	453.16/43.70/5.75	472.98/58.05/24.10
	S6	474.34/39.79/0.01	578.76/65.86/0.04	444.29/32.71/4.79	450.75/45.00/22.57
	S7	642.87/171.71/< 0.01	672.82/90.22/0.06	608.56/95.23/8.37	569.66/97.37/36.16

(continued on next page)

Table C.10 (continued).

Ins.	Sce.	Metrics (New Cmax/ADST/CPU Time (s))			
		Pure-RSR	A ² -RSR	ERRE	CRS
LA01	S1	675.12/11.97/< 0.01	685.87/14.76/< 0.01	630.76/4.94/0.27	654.88/13.06/0.97
	S2	782.73/41.70/< 0.01	712.93/49.97/0.01	666.69/30.37/0.34	684.83/38.52/1.08
	S3	958.95/182.78/< 0.01	801.76/106.73/< 0.01	742.74/86.58/0.42	738.54/108.52/1.21
	S4	821.60/72.19/< 0.01	765.12/64.88/< 0.01	697.36/35.86/0.66	741.37/57.53/2.19
	S5	984.70/243.82/< 0.01	841.52/147.92/< 0.01	770.18/101.25/0.81	809.06/133.88/2.39
	S6	870.69/102.11/0.01	822.28/118.97/< 0.01	722.24/66.83/0.72	755.59/102.35/2.25
	S7	1414.52/434.63/< 0.01	1198.05/259.50/< 0.01	1182.03/230.94/1.18	1281.61/337.22/3.67
LA08	S1	863.68/11.24/< 0.01	877.14/12.89/< 0.01	847.01/4.34/0.39	822.02/17.83/1.60
	S2	954.06/47.05/< 0.01	960.87/81.08/< 0.01	843.67/45.70/0.56	878.69/59.85/1.78
	S3	1109.12/194.73/< 0.01	1103.19/193.47/< 0.01	878.47/83.37/0.65	926.68/130.82/1.97
	S4	1009.65/88.57/< 0.01	1017.23/85.20/0.01	883.44/54.43/1.13	940.70/74.97/3.52
	S5	1141.29/228.90/< 0.01	1149.09/217.67/< 0.01	928.87/119.90/1.37	975.53/166.29/3.98
	S6	1066.83/121.45/< 0.01	1133.15/207.47/< 0.01	912.79/85.77/1.18	956.22/136.15/3.69
	S7	1548.77/471.45/< 0.01	1448.56/316.54/0.02	1326.35/297.35/2.09	1361.16/319.46/5.88
LA11	S1	1148.48/7.48/< 0.01	1222.03/21.17/< 0.01	1127.01/5.80/0.55	1141.65/24.44/2.40
	S2	1248.53/49.20/< 0.01	1248.70/100.26/0.01	1145.88/71.85/0.80	1159.81/71.65/2.64
	S3	1474.82/238.63/< 0.01	1329.81/216.59/< 0.01	1193.47/124.90/0.99	1244.21/157.95/2.89
	S4	1282.95/71.96/0.02	1323.87/125.87/0.01	1179.84/75.31/1.60	1230.61/82.14/5.28
	S5	1456.11/243.08/< 0.01	1422.43/246.15/0.02	1245.17/159.99/1.99	1287.76/190.79/5.91
	S6	1370.00/126.19/< 0.01	1381.60/229.49/< 0.01	1217.92/116.03/1.59	1247.61/154.37/5.35
	S7	1914.39/536.70/< 0.01	1703.10/384.57/0.02	1616.59/329.34/2.97	1619.66/373.96/8.91
LA20	S1	832.89/7.29/< 0.01	810.48/10.68/< 0.01	764.32/2.64/0.59	818.53/7.52/2.08
	S2	901.71/44.52/< 0.01	892.17/31.99/< 0.01	807.78/18.85/0.84	838.26/28.61/2.37
	S3	1142.31/222.43/< 0.01	1021.72/86.48/< 0.01	859.92/56.19/1.04	932.96/92.89/2.68
	S4	977.88/70.62/< 0.01	951.16/44.02/0.01	793.80/18.29/1.57	890.31/52.60/4.88
	S5	1093.18/197.80/< 0.01	1079.87/106.72/0.01	877.61/64.36/2.00	969.57/121.43/5.31
	S6	1072.30/126.91/< 0.01	1066.58/89.01/0.01	838.51/44.13/1.70	946.23/98.24/4.95
	S7	1565.72/520.84/< 0.01	1243.10/134.24/0.02	1225.31/172.39/2.91	1250.99/216.61/7.91
LA21	S1	966.54/9.81/< 0.01	1006.82/12.89/< 0.01	953.80/7.04/0.90	971.26/12.03/3.69
	S2	1074.42/52.91/< 0.01	1158.09/50.61/< 0.01	1007.10/40.18/1.27	1072.17/56.97/4.21
	S3	1289.28/219.86/< 0.01	1207.78/121.07/< 0.01	1061.59/83.50/1.64	1164.70/117.26/4.55
	S4	1138.86/79.30/< 0.01	1246.27/72.19/0.02	1037.39/42.82/2.50	1120.08/73.50/8.43
	S5	1274.18/212.52/< 0.01	1266.41/135.75/0.02	1103.85/100.05/3.36	1195.93/140.24/9.17
	S6	1217.17/129.85/< 0.01	1262.98/134.86/0.02	1059.28/76.87/2.64	1139.13/111.33/8.51
	S7	1729.71/443.31/0.02	1419.73/179.82/0.03	1351.25/147.50/4.54	1467.59/247.09/13.67
LA26	S1	1213.81/11.89/< 0.01	1264.74/17.05/< 0.01	1177.19/7.58/1.18	1210.35/13.44/5.79
	S2	1303.96/50.98/< 0.01	1446.47/100.58/0.01	1232.84/41.03/1.70	1278.37/65.19/6.62
	S3	1479.87/210.00/< 0.01	1501.77/224.16/0.02	1274.70/93.92/2.47	1379.38/144.76/7.20
	S4	1328.34/70.06/0.01	1443.35/76.91/0.02	1273.17/51.35/3.68	1336.96/89.22/13.38
	S5	1466.09/214.71/< 0.01	1535.72/247.83/0.02	1336.42/119.25/4.82	1397.25/148.95/14.34
	S6	1479.35/137.77/< 0.01	1526.67/238.81/0.02	1306.61/100.65/3.88	1391.75/156.04/13.49
	S7	1880.06/489.79/< 0.01	1683.81/234.44/0.03	1684.35/245.61/7.11	1612.11/275.30/21.76
LA31	S1	1645.31/7.70/< 0.01	1713.15/15.68/< 0.01	1606.51/13.91/2.07	1643.53/19.24/11.69
	S2	1731.43/48.49/< 0.01	1842.81/115.17/0.01	1709.97/80.32/3.06	1735.39/80.74/12.79
	S3	1926.92/251.87/< 0.01	1998.37/296.52/0.03	1749.44/144.29/4.30	1826.97/185.24/14.33
	S4	1826.05/94.08/< 0.01	1877.18/131.68/0.03	1729.38/87.18/6.20	1765.65/78.96/25.66
	S5	1911.04/222.19/< 0.01	2019.33/279.34/0.04	1798.18/160.02/8.27	1877.27/209.55/28.57
	S6	1860.09/125.24/< 0.01	1997.65/263.02/0.03	1768.03/139.38/6.43	1851.09/196.72/26.46
	S7	2381.82/480.44/< 0.01	2097.89/298.98/0.05	2140.68/345.21/13.20	2085.73/334.56/43.06
LA38	S1	1145.98/6.19/< 0.01	1158.01/5.62/< 0.01	1101.28/3.87/1.34	1145.48/5.97/6.64
	S2	1233.94/56.52/< 0.01	1264.38/42.03/< 0.01	1192.87/42.01/2.47	1240.04/50.55/7.70
	S3	1411.60/216.88/< 0.01	1402.28/106.54/0.01	1273.97/93.31/3.00	1326.56/119.33/8.52
	S4	1267.79/69.83/< 0.01	1287.78/53.61/0.02	1218.87/42.24/4.60	1286.25/69.90/15.60
	S5	1426.43/226.74/< 0.01	1388.72/99.49/0.02	1287.77/119.68/6.15	1370.20/150.22/16.79
	S6	1330.08/112.97/< 0.01	1387.18/96.29/0.02	1257.19/79.23/4.96	1337.31/123.45/15.07
	S7	1823.56/434.89/< 0.01	1561.57/121.04/0.03	1619.62/195.36/8.65	1598.14/230.35/24.11
LAR01	S1	112.88/2.32/< 0.01	101.23/1.47/< 0.01	91.09/0.06/0.46	101.43/2.09/1.29
	S2	144.14/7.97/< 0.01	104.74/5.13/0.01	112.35/0.60/0.54	106.83/5.49/1.41
	S3	229.62/35.12/< 0.01	147.48/8.16/0.02	145.36/2.31/0.73	154.11/16.12/1.49
	S4	148.25/14.88/< 0.01	113.62/7.04/0.01	114.52/1.57/1.06	117.79/7.73/2.79
	S5	231.97/36.90/< 0.01	150.02/9.40/0.02	153.60/4.76/1.21	149.73/15.25/2.21
	S6	156.24/16.90/< 0.01	102.27/7.45/< 0.01	96.64/3.01/0.71	108.27/11.53/1.48
	S7	352.35/65.47/< 0.01	299.88/10.04/0.01	292.65/4.21/1.26	269.74/32.83/3.11

(continued on next page)

Table C.10 (continued).

Ins.	Sce.	Metrics (New Cmax/ADST/CPU Time (s))			
		Pure-RSR	A ² -RSR	ERRE	CRS
LAR02	S1	141.40/2.18/< 0.01	137.56/1.96/0.02	121.07/0.11/0.52	136.06/2.10/2.88
	S2	184.30/7.10/< 0.01	141.70/4.52/< 0.01	131.58/0.77/0.89	138.91/5.91/3.11
	S3	251.83/22.49/< 0.01	163.40/13.73/0.01	172.52/3.80/1.13	171.36/19.13/3.27
	S4	189.04/12.20/< 0.01	148.63/6.47/0.02	133.34/1.58/1.59	155.80/9.23/6.20
	S5	251.76/29.57/< 0.01	166.47/13.07/0.03	149.23/5.94/1.71	165.87/19.67/6.07
	S6	183.44/18.03/< 0.01	147.95/13.75/0.01	143.73/5.09/1.64	152.55/16.24/6.20
	S7	397.71/79.01/< 0.01	316.25/15.88/0.03	320.00/4.07/2.40	314.10/41.31/8.45
LAR03	S1	251.90/1.54/< 0.01	249.38/2.44/0.02	236.74/0.47/1.54	249.19/1.94/10.46
	S2	289.42/6.83/< 0.01	251.95/12.60/0.03	236.32/3.33/2.14	255.40/12.58/11.25
	S3	369.75/37.66/< 0.01	270.71/33.68/0.04	257.35/9.34/3.35	279.89/42.65/11.91
	S4	298.85/13.16/< 0.01	266.81/14.64/0.05	245.27/3.51/4.42	265.23/8.16/22.27
	S5	361.22/40.39/< 0.01	273.31/40.41/0.07	265.57/11.11/4.11	279.54/40.47/23.90
	S6	303.81/18.13/< 0.01	271.05/32.92/0.05	249.60/6.06/4.46	269.86/43.25/23.31
	S7	510.29/78.98/< 0.01	393.72/40.20/0.10	402.34/9.39/6.74	407.30/58.86/35.34
MT20	S1	1103.47/9.41/< 0.01	1106.34/17.17/< 0.01	1071.30/7.76/0.57	1073.02/17.76/2.36
	S2	1210.43/50.58/< 0.01	1201.67/90.80/< 0.01	1082.45/39.89/0.72	1120.08/59.81/2.62
	S3	1367.86/163.93/< 0.01	1343.94/216.50/< 0.01	1134.58/106.01/0.96	1177.95/150.57/2.94
	S4	1259.19/78.67/< 0.01	1225.72/91.87/0.01	1137.34/67.40/1.63	1165.46/81.14/5.35
	S5	1399.15/224.36/< 0.01	1445.85/288.99/0.01	1171.27/141.66/1.95	1220.66/166.97/5.91
	S6	1293.09/104.07/< 0.01	1407.33/281.36/0.01	1171.24/99.11/1.58	1193.10/177.69/5.46
	S7	1822.92/478.06/0.01	1724.38/356.29/0.02	1500.33/242.00/2.76	1538.34/351.24/8.92

References

Abumaizar, R. J., & Svestka, J. A. (1997). Rescheduling job shops under random disruptions. *International Journal of Production Research*, 35(7), 2065–2082.

Angel-Bello, F., Vallikavungal, J., & Alvarez, A. (2020). Simultaneous consideration of sequence-dependent and position-dependent setup times in single machine scheduling problems. *Journal of Mathematics and Computer Science*, 10(5), 2033.

Angel-Bello, F., Vallikavungal, J., & Alvarez, A. (2021). Fast and efficient algorithms to handle the dynamism in a single machine scheduling problem with sequence-dependent setup times. *Computers & Industrial Engineering*, 152, Article 106984.

Aytug, H., Lawley, M. A., McKay, K., Mohan, S., & Uzsoy, R. (2005). Executing production schedules in the face of uncertainties: A review and some future directions. *European Journal of Operational Research*, 161(1), 86–110.

Behnke, D., & Geiger, M. J. (2012). Test instances for the flexible job shop scheduling problem with work centers.

Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3), 157–183.

Chang, X., Jia, X., Fu, S., Hu, H., & Liu, K. (2023). Digital twin and deep reinforcement learning enabled real-time scheduling for complex product flexible shop-floor. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 237(8), 1254–1268.

Chen, R., Yang, B., Li, S., & Wang, S. (2020). A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers & Industrial Engineering*, 149, Article 106778.

Cowling, P., & Johansson, M. (2002). Using real time information for effective dynamic scheduling. *European Journal of Operational Research*, 139(2), 230–244.

Dauzère-Pérès, S., Ding, J., Shen, L., & Tamssaouet, K. (2024). The flexible job shop scheduling problem: A review. *European Journal of Operational Research*, 314(2), 409–432.

Fan, J., Zhang, C., Tian, S., Shen, W., & Gao, L. (2024). Flexible job-shop scheduling problem with variable lot-sizing: An early release policy-based matheuristic. *Computers & Industrial Engineering*, 193, Article 110290.

Fang, W., Zhang, H., Qian, W., Guo, Y., Li, S., Liu, Z., Liu, C., & Hong, D. (2023). An adaptive job shop scheduling mechanism for disturbances by running reinforcement learning in digital twin environment. *Journal of Computing and Information Science in Engineering*, 23(5), Article 051013.

Feng, Z., Zou, Z., & Liang, X. (2025). Solving machine overload for re-scheduling of dynamic flexible job shop by adaptive tripartite game theory-based genetic algorithm. *Swarm and Evolutionary Computation*, 96, Article 101938.

Gui, Y., Tang, D., Zhu, H., Zhang, Y., & Zhang, Z. (2023). Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. *Computers & Industrial Engineering*, 180, Article 109255.

Gupta, S., & Jain, A. (2022). Impact of preventive maintenance and machine breakdown on performance of stochastic flexible job shop scheduling with setup time. *Journal of Scientific & Industrial Research*, 81(11), 1195–1203.

Hammami, N. E. H., Lardeux, B., Hadj-Alouane, A. B., & Jridi, M. (2025). Design and calibration of a DRL algorithm for solving the job shop scheduling problem under unexpected job arrivals. *Flexible Services and Manufacturing Journal*, 37(1), 125–156.

Herrmann, J. W. (2006). Rescheduling strategies, policies, and methods: Using the rescheduling framework to improve production scheduling. *Handbook of Production Scheduling*, 135–148.

Hurink, J., Jurisch, B., & Thole, M. (1994). Tabu search for the job-shop scheduling problem with multi-purpose machines. *Operations-Research-Spektrum*, 15(4), 205–215.

Jiang, B., Ma, Y., Chen, L., Huang, B., Huang, Y., & Guan, L. (2023). A review on intelligent scheduling and optimization for flexible job shop. *International Journal of Control, Automation and Systems*, 21(10), 3127–3150.

Kacem, I., Hammadi, S., & Borne, P. (2002). Approach by localization and multiobjective evolutionary optimization for flexible job-shop scheduling problems. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 32(1), 1–13.

Kulcsár, G., Kulcsárné Forrai, M., & Cservenák, Á. (2025). Objective functions for minimizing rescheduling changes in production control. *Automation*, 6(3), 30.

Leus, R., & Herroelen, W. (2007). Scheduling for stability in single-machine production systems. *Journal of Scheduling*, 10(3), 223–235.

Li, X., & Gao, L. (2016). An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem. *International Journal of Production Economics*, 174, 93–110.

Li, J., Zhang, W., & Li, J. (2025). Solving multi-objective energy-efficient flexible job shop problems by a dual-level NSGA-II algorithm. *Memetic Computing*, 17(2), 1–29.

Liu, L., Guo, K., Gao, Z., Li, J., & Sun, J. (2022). Digital twin-driven adaptive scheduling for flexible job shops. *Sustainability*, 14(9), 5340.

Liu, C.-L., Tseng, C.-J., & Weng, P.-H. (2024). Dynamic job-shop scheduling via graph attention networks and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 20(6), 8662–8672.

Liu, C.-L., Weng, P.-H., & Tseng, C.-J. (2025). Harnessing heterogeneous graph neural networks for dynamic job-shop scheduling problem solutions. *Computers & Industrial Engineering*, 203, Article 111060.

Ma, J., Gao, W., & Tong, W. (2025). A deep reinforcement learning assisted adaptive genetic algorithm for flexible job shop scheduling. *Engineering Applications of Artificial Intelligence*, 149, Article 110447.

Maierhofer, A., Trojahn, S., & Ryll, F. (2025). Dynamic scheduling and preventive maintenance in small-batch production: A flexible control approach for maximising machine reliability and minimising delays. *Applied Sciences*, 15(8), 4287.

Moghadam, A. M., Wong, K. Y., & Piroozfard, H. (2014). An efficient genetic algorithm for flexible job-shop scheduling problem. In *2014 IEEE international conference on industrial engineering and engineering management* (pp. 1409–1413). IEEE.

Ouelhadj, D., & Petrovic, S. (2009). A survey of dynamic scheduling in manufacturing systems. *Journal of Scheduling*, 12(4), 417–431.

Peng, C., Zhang, Z., Liao, T. W., Zhao, H., & Cai, Y. (2025). A hybrid approach for the dynamic flexible job shop scheduling problem considering machine failures. *Journal of Scheduling*, 1–18.

Pleier, F., & Schlip, J. (2024). Predictive-reactive scheduling to increase robustness of production systems. In *2024 4th international conference on electrical, computer, communications and mechatronics engineering* (pp. 1–6). IEEE.

Priore, P., Gómez, A., Pino, R., & Rosillo, R. (2014). Dynamic scheduling of manufacturing systems using machine learning: An updated review. *AI Edam*, 28(1), 83–97.

Pujawan, I. N. (2004). Schedule nervousness in a manufacturing system: A case study. *Production Planning and Control*, 15(5), 515–524.

Ren, F., & Liu, H. (2024). Dynamic scheduling for flexible job shop based on MachineRank algorithm and reinforcement learning. *Scientific Reports*, 14(1), Article 29741.

- Serrano-Ruiz, J. C., Mula, J., & Poler, R. (2024). Job shop smart manufacturing scheduling by deep reinforcement learning. *Journal of Industrial Information Integration*, 38, 1–27.
- Tang, H., Xiao, Y., Zhang, W., Lei, D., Wang, J., & Xu, T. (2024). A DQL-NSGA-III algorithm for solving the flexible job shop dynamic scheduling problem. *Expert Systems with Applications*, 237, Article 121723.
- Vieira, G. E., Herrmann, J. W., & Lin, E. (2003). Rescheduling manufacturing systems: A framework of strategies, policies, and methods. *Journal of Scheduling*, 6(1), 39–62.
- Wang, C., Wei, L., Sun, H., & Hu, Z. (2025). Knowledge-driven inverse diffusion prediction algorithm for flexible job shop scheduling problem considering transportation resources and multiple breakdowns. *Swarm and Evolutionary Computation*, 96, Article 101979.
- Wei, L., He, J., Guo, Z., & Hu, Z. (2023). A multi-objective migrating birds optimization algorithm based on game theory for dynamic flexible job shop scheduling problem. *Expert Systems with Applications*, 227, Article 120268.
- Wu, R., Zheng, J., & Yin, X. (2025). Dynamic scheduling for multi-objective flexible job shops with machine breakdown by deep reinforcement learning. *Processes*, 13(4), 1246.
- Yan, Q., Wang, H., & Wu, F. (2022). Digital twin-enabled dynamic scheduling with preventive maintenance using a double-layer Q-learning algorithm. *Computers & Operations Research*, 144, Article 105823.
- Yi, W., Deng, Q., Wang, B., Chen, Y., & Pei, Z. (2025). Dynamic flexible job shop scheduling problem considering multiple types of dynamic events. *International Journal of Production Research*, 1–18.
- Zarrouk, R., Bennour, I. E., & Jemai, A. (2019). A two-level particle swarm optimization algorithm for the flexible job shop scheduling problem. *Swarm Intelligence*, 13(2), 145–168.
- Zhang, F., Li, R., & Gong, W. (2024). Deep reinforcement learning-based memetic algorithm for energy-aware flexible job shop scheduling with multi-AGV. *Computers & Industrial Engineering*, 189, Article 109917.
- Zhang, S., & Wong, T. N. (2017). Flexible job-shop scheduling/rescheduling in dynamic environment: A hybrid MAS/ACO approach. *International Journal of Production Research*, 55(11), 3173–3196.
- Zhao, L., Fan, J., Zhang, C., Shen, W., & Zhuang, J. (2024). A DRL-based reactive scheduling policy for flexible job shops with random job arrivals. *IEEE Transactions on Automation Science and Engineering*, 21(3), 2912–2923.
- Zhou, Y., Zheng, T., Hu, M., Zhang, J., & Yuan, M. (2023). Deep reinforcement learning for large-scale flexible job-shop dynamic scheduling problem. In *2023 IEEE smart world congress* (pp. 879–884). IEEE.